

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 19:40:17
Файлов исходного кода: 87
Суммарный объём кода: 1.20 MB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасика [id: aho-corasick]

```
24. src/data/vizStepBus.ts
25. src/data/vizSync.ts
```

Билеты (учебный контент)

```
26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts
```

Интерактивные компоненты и симуляторы

```
30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/DfsBridgesSimulator.tsx
37. src/components/DijkstraViz.tsx
38. src/components/DPVisualizer.tsx
39. src/components/EulerSimulator.tsx
40. src/components/ExpressQuiz.tsx
41. src/components/FloydViz.tsx
42. src/components/GraphTraversalViz.tsx
43. src/components/HeapViz.tsx
44. src/components/hints/demoKit.tsx
45. src/components/hints/demosExtra.tsx
46. src/components/hints/demosGraph.tsx
47. src/components/hints/demosStruct.tsx
48. src/components/hints/MiniDemo.tsx
49. src/components/hints/SimulatorModal.tsx
50. src/components/hints/TermPopover.tsx
51. src/components/JohnsonViz.tsx
52. src/components/KosarajuViz.tsx
53. src/components/KruskalSimulator.tsx
54. src/components/MnemonicCards.tsx
55. src/components/MobileToc.tsx
56. src/components/Navbar.tsx
57. src/components/PlanarityDemo.tsx
```

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

```
 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```

Универсальное пособие • полный исходный код

```
-----
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ,,,
+

## `./.github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml
name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (
|
→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "*"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
```

```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
На Ubuntu политика ImageMagick иногда запре
разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${public}/export/full\_code\_all\_pages  
 echo "| \`\${public}/export/full\_code\_all\_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```

 <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

```
`public/favicon.svg`
```

```
Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
</svg>  
....
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
....
```

```
`src/App.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from  
import { chapters } from "../data/content";
```



```

    }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
            {/* Сайдбар только на десктопе — на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0">
              <Sidebar selectedChapterId={activeChapterId}>
            </div>

            <main id="main-content" className="flex-1 min-w-0">
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between">
                <div className="min-w-0">
                  <div className="text-[10px] uppercase">
                    {activeChapter.category || "Универсальное пособие"}
                  </div>
                  <h2 className="text-base sm:text-xl font-medium">
                    {activeChapter.title}
                  </h2>
                </div>
                <ChapterNav prev={prev} next={next} index={index}>
              </div>
            </main>
          </div>
        </>
      ) : (
        <ChapterList>
      )}
    </Navbar>
  </div>
);

```

```

        </footer>
    </div>
  );
}
....

### Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
 </>
 </main>
) : (
 <main>
 <ChapterNav />
 <ChapterView />
 </main>
)
}

```

```

<div className="flex items-center gap-3 w-full" >
 <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1" >
 <h1 className="text-xl font-extrabold text-slate-900" >
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium" >
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <CodeIcon className="w-4 h-4" /> Симулятор
 </button>
</div>

```

```


 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

```

Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
 }
 return runtimePromise;
}
```

```

 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.end`
 setRuntimeMessage("Не удалось выполнить код – про
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>
 </div>
 </div>
 </main>

```

```

 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
 </div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

 </div>
 <pre className="min-h-[360px] max-h-[560px]
 x] leading-6 text-slate-300">
 {output}

```

Универсальное пособие • полный исходный код

```

+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`
+
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) => void }) => void
+ setStderr: (options: { batched: (message: string) => void }) => void
+ setStdin: (options: { stdin: () => string | number | null }) => void
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
```



```

+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1.
+ <section className="rounded-2xl border borde
+ <div className="flex flex-wrap items-cente
+ <div className="flex items-center gap-2"
+ <span className="w-2.5 h-2.5 rounded-f
+ <span className="text-sm font-bold tex
+ <span className="text-[11px] text-slate

```

```

+
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin: 10px 0;}}>
+ <label className="block px-4 pt-3 text-slate-600 font-medium">Ввод для input() • по одной строке на строку</label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border border-slate-800 rounded-md px-4 py-3 text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 20px; margin: 10px 0;}}>
+ <div className="flex items-center justify-between" style={{border: 1px solid #444; border-radius: 10px; padding: 5px; margin-bottom: 10px;}}>
+ <div className="flex items-center gap-2" style={{border: 1px solid #444; border-radius: 10px; padding: 5px; flex: 1;}}>
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #444; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin-top: 10px;}}>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #444; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin-bottom: 10px;}}>

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
},
server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
},
preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
},
});
```
```

Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
```

Универсальное пособие • полный исходный код

- 
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
  3. **\*\*Строгая структура контента:\*\***
    - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
    - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
    - Обязательная сетка аналогий (2 колонки: неформальный текст и код)
    - "Душные" детали (код, формулы, сложные доказательства)
  4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

---

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

---

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "version": "1.0.0",
 "description": "A reader for the ExamPrep Remix Remix book.",
 "keywords": ["remix", "remix", "exam", "prep", "reader"],
 "author": "Remix Remix",
 "license": "MIT",
 "repository": {
 "type": "git",
 "url": "https://github.com/remix-remix/remix-remix-exam-prep-reader"
 },
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview"
 },
 "dependencies": {
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-router-dom": "^6.14.2",
 "remix": "^2.10.0"
 },
 "devDependencies": {
 "@remix-run/dev": "^2.10.0",
 "@types/react": "^18.2.0",
 "@types/react-dom": "^18.2.0",
 "@types/react-router-dom": "^6.14.2",
 "typescript": "^5.0.0",
 "vite": "^4.3.9"
 }
}
```

```
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

---

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5174
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- \* **\*\*Содержание\*\*** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

## ## Структура

...

src/

- |                    |                                           |
|--------------------|-------------------------------------------|
| App.tsx            | – оболочка, привязка глав к визуализации  |
| components/        | – интерактивные симуляторы (Действия)     |
| data/              | – контент пособия (главы/билеты)          |
| scripts/export/    | – пайплайн код → txt → png → pdf          |
| public/export/     | – сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | – автоматизация                           |

...

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```

> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет         | Тема                           | Статус                                  |
|---------------|--------------------------------|-----------------------------------------|
| ---           | ---                            | ---                                     |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии. |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть блок аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть блок аналогии. |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динамическое программирование\*\***

|    |                                               |   |       |
|----|-----------------------------------------------|---|-------|
| 9  | 9. Графы. Топологическая сортировка           | 1 | -     |
| 10 | 10. Графы. Компоненты сильной связности (SCC) |   |       |
| 12 | 12. Графы. Точки сочленения                   | 1 | -   - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 | -     |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 |       |
| 16 | 16. Графы. Кратчайший путь. Флойд             | - | -     |
| 17 | 17. Графы. Алгоритм Джонсона                  | - | -     |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 | -     |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 | -     |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 | -     |
| 21 | 21. Префикс-функция (π), КМП                  | 4 | -   - |
| 22 | 22. Z-функция                                 | 4 | -   - |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 | -   - |
| 24 | 24. Классы сложности, сведение задач          | 4 | -   - |
| -  | Обзор: Кратчайшие пути и Графы                | - | -   - |
| -  | Алгоритмы на карте                            | - | -   - |
| -  | Эйлер: Путь ≠ Цикл                            | - | -   - |
| -  | Мосты в графах: код + визуализация            | 1 | -   - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
```



```

import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz)?.scrollIntoView();
 });
 });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
 {activeTab === "guide" ? (
 <div>
 <MobileToc

```

```

 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>
) : (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
)}

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

{compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>

```

```
* , чтобы на
* повесить всплывающую карточку (как превью ссылок в В
*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 }
 }
}
```

```
 try {
 annotateTerms(el);
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 KB

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
```

```

 from {
 opacity: 0;
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
 }

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в В
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

```

```
.chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
}
.chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/87: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

---



## Аналогия 2: Ступени

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приори
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`,
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <!-- 0. Определение -->
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-lg sm:text-xl font-mono">
 поиска без
 : серия поворотов, которая поднимает v в кор
 <p class="text-sm text-slate-400 text-c">
 т – поворот.</p>
 </div>

```

```

 <ul class="list-disc list-inside text-slate-300">
 Важно!
 каждой вставки чинят дерево, чтобы оно не
 /li>
 Важно!
 менно быть кривым. Но каждый раз, когда мы
 ровнее становится.

 </div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 Аналогия 1: стопка бумаг на столе
 </div>
 <p class="text-slate-300 text-sm">У тебя
 скиваешь его и, поработав, кладёшь Важно!
 неделю самые нужные бумаги сами собой оказыва
 ет ровно это: «взял → положил наверх».</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
 Аналогия 2: тропинка через газон
 </div>
 <p class="text-slate-300 text-sm">Ты идёшь по дорожке, ко
 ода дорожка сама укорачивалась вдвое: чем чаще
 оплачивает все следующие. Пойдёшь один раз
 </p>
</div>

```

<!-- 3. Поворот -->

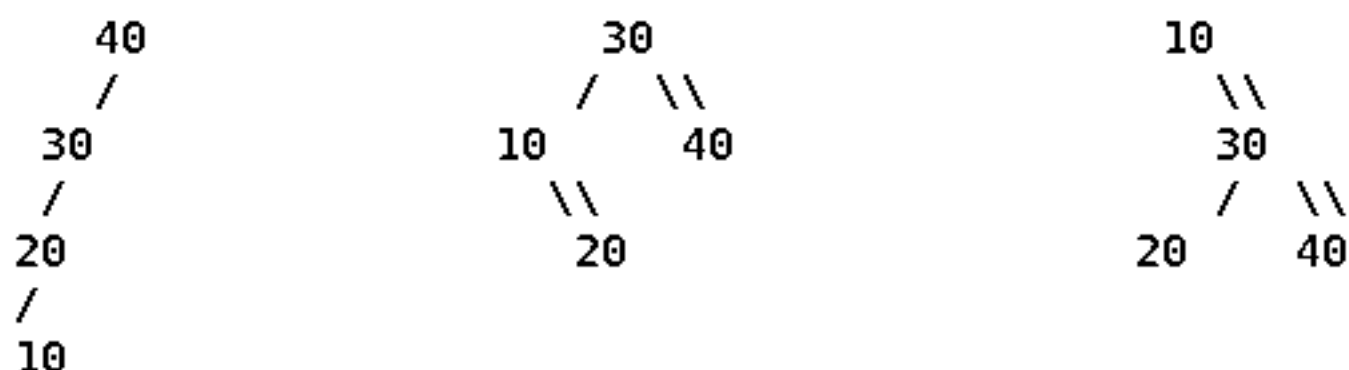
```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-4">Поворот
 <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки и

```

[illegible]

```
<p class="text-slate-300 text-sm mb-4">Дерево
3 шага, а потом делаем Splay(10).</p>
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-
0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```



итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2
<h3 class="text-lg font-bold text-white mb-3">
<p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag
ах идея такая:</p>
<div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">H
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">д
```

```

// один поворот: x встаёт на место своего родителя
function rotate(x) {
 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда нет
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
 }
 return x; // теперь x —
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-2 border-
 <summary class="p-4 font-bold text-slate-400"
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (и
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Последний
 енный или преемник). Splay делают всегда, да
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-2">Почему?
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-2">Как?
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-2">Пример
 <p class="text-slate-400">Treap дер
 play не хранит вообще ничего и держит балан
 <p>
 </div>
 </div>
</details>

</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 }

```

```

 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white"
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

 description: "Разбиение орграфа на макро-вершины. А
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 БЫХ направлениях по односторонним улицам. Как это отразится
 у на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутые",
 content: `

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-rose-300 italic"
 <p class="text-xl font-mono text-white"
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

```

```

 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-6">
изкому племени для объединения. К вечеру пл

```

```

 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Сложность
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-purple-400">Сведение
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Сведение (Reduction)
 <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP —
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">ь отличный переводчик на английский (Сведени
 аче B, то B — это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
`

```

ин раз. Вершины повторять можно.</p>

**Эйлер**

<div class="p-3 bg-slate-950 rounded

<p class="text-sm text-blue-300

**Π**

**U**

<p class="text-slate-300 text-sm">

**Билет 13 (Различие пути и цикла**

```

* <span class="text-green-400 font-
шел-вошёл-вышел).
```

```

* <span class="text-yellow-400 font
```

</p>

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap
```

```
<div class="bg-slate-800 p-6 rounded-lg bor
```

```
<div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
```

Путь: Нормальная прогулка

<p class="text-slate-300 text-sm mb-4">

Ты вышел из дома <strong>(нечётная)</strong>  
талый пришёл в бар <strong>(вторая нечётная)</strong>  
Домой вернуться не смог, потому что

</p>

<div c

<div class="bg-slate-900 p-6 rounded-lg border

```
<div class="absolute -top-3 left-4 bg-rose-500 shadow-md bg-slate-950">
```

Цикл: Гамильтонов синдром (болезнь)

`<div class="text-slate-300 text-sm mb-4">`

```
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Мосты
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v]
 <p><strong class="text-white">low[v]
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold text-white">Мост: Единственная веревка
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся — граф р-связен.
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только веревка.
 Если верёвка (ребро v-u) оборвётся — граф р-связен.
 </p>
 </div>
```



```

int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `<code class="text-white">V=5, -white">10 &gt; 9</code>` – БАХ, непланарен!

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забу

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `<code class="text-white">V=6 xt-white">9 ≤ 12</code>` – проходит, но он в

Почему? Потому что у двудольного гр

Отсюда более строгое ограничение: `<`

```
<code class="text-white">9 > 8</code>
```

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скры

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 spac
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен.  $V = 5$ ,  $E = 10$ .

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекроют (удалят вершину) – районы будут
 хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень) */
 func: string;
 /** Локальные переменные: имя → герг (усечённый). */
 locals: Record<string, string>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Текст последней необработанной ошибки (или пустая) */
 error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
 return `import sys as __sys, json as __json`
```

```

 })
 OUT
 }

 /**
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как амбер-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
 export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
): Record<string, boolean> {
 const out: Record<string, boolean> = {};
 for (const k of Object.keys(cur)) {
 out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
 }
 return out;
 }

 /** Базовое имя из карточки переменной: "tin[v]" → "tin"
 export const baseVarName = (name: string): string => name

 /**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
 export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
 }

```

```

 short:
 "Волна от источника: сначала все соседи, потом со
formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
complexity: "O(V + E)",
demo: "bfs",
simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину",
title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
complexity: "O(V + E)",
demo: "dfs",
simulator: "stack-dfs",
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
formal:
 "Полное бинарное дерево в массиве: дети узла i ле
complexity: "вставка/извлечение O(log n), «посмотре
demo: "heap",
simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"
title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь све

```

```

 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то обход»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксную ширину, то есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень) и затем кладём с в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки"],
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла к строке.",
 formal: "link(v) = go(link(parent(v)), ch). Считает суффикс",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топология"],
 title: "Топологическая сортировка",
 short:

```



```

 "Заранее считаем ответ для всех отрезков длины 1,
 м.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
 },
 {
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь массив кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
 },
 {
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается на две части.",
 formal: "Ребро (u, v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где $\text{tin}[u]$ — время захода в вершину u при обходе в глубину (и $\text{low}[v]$ — минимальное tin среди всех вершин, достижимых из v путём, не выходящим за пределы поддерева, корнем которого является u и не содержащим ребро в родителя — нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
 },
 {
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точки разрыва"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапсирует граф.",
 formal: "Для не-корня: v — точка сочленения $\iff \exists$ ребро (u, v) такое, что $\text{low}[v] \geq \text{tin}[u]$ ",
 demo: "articulation",
 }

```

```

 ые в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полн
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»:
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h – ра
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
},
{
 id: "scc",
 labels: ["компонента сильной связности", "компонент
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до кажд
 formal: "Косарайю: DFS с сохранением порядка выхода
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу
 "префикс", "префикса", "суффиксом", "суффикс", "с
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начал
 от кусок.",
 formal:

```

```

 short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» — по с
 formal:
 "Дейкстра — неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешевое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
 title: "Эйлеров путь и цикл",
 short:

```

```

 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Список",
 },
 {
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс",
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший н
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали н
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
 },
 {
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман",
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбра
 ",
 formal:
 "После k -й итерации гарантированно верны все крат
 ось — есть отрицательный цикл.",
 complexity: " $O(V \cdot E)$ ",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k ",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки

```

```

 "Сортируем все рёбра по весу и берём подряд те, ч
formal:
 "Сортировка $O(E \log E)$ + E операций union/find. C
complexity: "O(E log E)",
demo: "mst",
simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Горо
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица
demo: "graph",
simulator: "graph-dfs-bfs",
},
{

```

```

 labels: [
 "включений-исключений", "включения-исключения", "
 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больш
 рый отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведён
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы – префиксные суммы O(1). Минимум без изменен
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{

```

```

 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 GLOSSARY.map((e) => ({ label: e.labels[0], id: e.id }))
 .sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
(нельзя поехать и заработать). Он всегда выбирает
 <div id="slot-chapter-image-dijkstra"
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
охождение улицы), Дейкстра сломается, так как
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg mb-4">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь (min
g V). Гарантирует корректность только для неориентированной
 </div>
```



```

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #f8d7da; border-radius: 10px; padding: 10px; margin: 10px 0;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 2px solid #f8d7da; border-radius: 10px; padding: 5px; margin: 5px 0;">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-0">
 г станет ЕЩЕ короче – значит мы попали во в
 </p>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #f8d7da; border-radius: 10px; padding: 10px; margin: 10px 0;">
 <h3 class="text-xl font-bold text-indigo-400">
 Отрицательный цикл

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 ...
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 минальные ссылки (term_link) – прямые мос
 p>
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b

```

```

 nodes[child].term_link = nodes[nodes[child].term_link];
 }

 q.push(child);
}
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
 </div>
</section>`
 }
];

```

```

 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлера пути нет – нужно удалять лишние рёбра",
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлеров
 ровно 2 нечётные или 0."
}
],
"bridges-code": [
 {
 question: "После DFS у нас $low[u] = 4$, а $tin[v] = 2$ ",
 options: [
 "Да, так как $low[u] (4) > tin[v] (2)$ ",
 "Нет, $low[u]$ должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: $low[u] > tin[v]$. В на
 },
 {
 question: "Для чего нужен массив $low[v]$?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: " $low[v]$ хранит минимальное время входа
 поиска мостов и точек сочленения."
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов",
 options: [
 " $O(V^2)$ – ищем мосты в квадрате",
 " $O(V + E)$ – каждый элемент один раз",
 " $O(2^V)$ – полный перебор"
],
 },

```

---

ФАЙЛ 24/87: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 113 | РАЗМЕР: 1.5 КБ

---

```
import { createContext, useContext, useEffect } from "react";

/**
 * Связь «компилятор ↔ открытая визуализация».
 *
 * Панель компилятора в режиме пошагового отладчика транслирует
 * текущего шага трассы вместе с id текущей главы; пошаговые
 * главы (через useVizStepSync) подхватывают его и прыгают к нужной
 * визуализации идёт в ногу с листингом (каждая исполняемая строка
 * = очередной шаг подсветки, см. vizSync.ts).
 */

export const VizChapterContext = createContext("");

const CHANNEL = "algo:viz-step";

export interface VizStepEventDetail {
 chapterId: string;
 step: number;
}

/** Транслировать текущий шаг отладчика визуализациям трассы */
export function emitVizStep(chapterId: string, step: number) {
 if (typeof window === "undefined") return;
 window.dispatchEvent(new window.CustomEvent<VizStepEventDetail>(CHANNEL, {
 detail: { chapterId, step },
 }));
}

/**
 * Подписка визуализации на шаги отладчика.
 * Вызывается внутри компонента демонстрации:
 * useVizStepSync(currentStep, setCurrentStep, steps.length);
 */
```

```

}

/** Сколько шаговых («помеченных») строк успело выполни
export function vizHitsAtTrace(
 steps: ReadonlyArray<{ line: number }>,
 idx: number,
 srcLines: readonly string[],
 markedContents: ReadonlySet<string>
): number {
 let hits = 0;
 for (let t = 0; t <= idx && t < steps.length; t++) {
 const content = srcLines[steps[t].line - 1];
 if (content !== undefined && markedContents.has(con
 }
 return hits;
}

/* — Выбор демонстрации внутри страницы (вкладки ID /
const DEMO_CHANNEL = "algo:viz-demo";

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас откр
export function emitVizDemo(chapterId: string, demoId:
 if (typeof window === "undefined") return;
 window.dispatchEvent(new window.CustomEvent<VizDemoEv
}

/** Подписка: панель компилятора следует за активной вк
export function onVizDemo(chapterId: string, cb: (demoId
 if (typeof window === "undefined") return () => {};
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>

```

```

variables: SyncVariable[];
/**
 * Минимальный исполняемый скелет для редактора: стро
 * шаг-в-шаг с подсветкой визуализации (без словесных
 */
code?: string;
/**
 * Номера строк ВНУТРИ code (с 1), каждое выполнение
 * шаг визуализации: по ним отладчик точно переводит
 * шаг демонстрации (иначе – грубо «строчный индекс =
 */
stepCodeLines?: number[];
/**
 * Может ли демонстрация страницы ходить за отладчиком
 * false/нет – интерактив руками (клики/наведение): ч
 */
stepDriven?: boolean;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем tr
 code: `n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; лист
 { name: "i", role: "индекс элемента массива; его
 { name: "v", role: "текущая вершина дерева отрезк
 { name: "l, r", role: "границы отрезка, за которы
 { name: "m", role: "середина отрезка m = (l + r)
],

```





```

kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас обрабатываем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)`,
 variables: [
 { name: "top", role: "вершина стека — то, откуда мы вышли"},
 { name: "v", role: "текущая вершина графа, которую мы обрабатываем"},
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepDriven: true,
 stepNote: "Шаг = dequeue вершины из головы очереди и enqueue соседа",
 code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
 variables: [
 { name: "front", role: "голова очереди — вершина, которую мы обрабатываем"},
 { name: "v", role: "вершина, извлечённая из очереди"},
 { name: "u", role: "сосед вершины v, которого кладем в очередь"},
],
},

```

```

 variables: [],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepDriven: true,
 stepNote: "Шаг = выход рекурсии из вершины v: она д
 code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v) # выход из v!

for v in adj:
 if v not in used:
 dfs(v)
print(order[::-1]) # разворот – ответ`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepDriven: true,
 stepNote: "Шаг = вершина из order (в обратном поряд
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {v: [] for v in adj}
for v in adj:
 for to in adj[v]:
 adjT[to].append(v) # транспонирование

order = [3, 2, 1, 0, 4] # фаза 1 уже дала порядок в

```

## Универсальное пособие • полный исходный код

```

code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверяя
 { name: "tin[v]", role: "время входа в вершину –
 { name: "low[v]", role: "минимум tin, куда можно
 { name: "timer", role: "глобальный счётчик времени
],
},
"euler-path-vs-cycle": {
 vizTitle: "Эйлеров путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине мар
 code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
0 → цикл · 2 → путь · иначе – нельзя`,
 variables: [
 { name: "path", role: "текущий маршрут – последов
 { name: "last", role: "последняя вершина пути – о
 { name: "deg(v)", role: "степень вершины: чётная
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5 и K3,3",
 stepNote: "Шаг = попытка перетащить вершину так, что
 code: `V, E = 5, 10 # K5 (K3,3: V=
F = 2 - V + E # формула: V - E + F = 2

```

```

 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepDriven: true,
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
 u, v, w = "S", "A", 5 # шаг: очередное ребро
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
 if i == 1:
 print(f"итерация {i}: {dist}")
 break`
 variables: [
 { name: "n", role: "число вершин графа – ровно n"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
 { name: "dist[v]", role: "расстояние; если обновлено, то"},
],
},
floyd: {
 vizTitle: "Флойд–Уоршелл",
 stepDriven: true,
 stepNote: "Шаг = инициализация матрицы, смена промежуточного",
 code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0],
 [INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):
 print(f"посредник k={k}")

```

```
code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C")]
edges.sort() # по весу
```

```
parent = {v: v for v in "ABC"} # DSU
```

```
def find(x):
 while parent[x] != x:
 x = parent[x]
 return x
```

```
mst = []
```

```
for w, u, v in edges: # шаг: очередное ребро
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
```

```
print(mst)`,
```

```
variables: [
 { name: "n", role: "число вершин; в остов войдёт n вершин"},
 { name: "m", role: "число рёбер, сортируем по весу"},
 { name: "i", role: "ребро сортируем по весу w; ра"},
 { name: "parent[v]", role: "DSU: кто представитель"},
 { name: "rank[v]", role: "высота дерева DSU – по"},
],
```

```
},
```

```
"mst-prim": {
 vizTitle: "Прим",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq
```

```
start = "A"
```

```
heap = [(0, start, "start")] # (вес, вершина, откуда)
```

```
w, v, parent = heapq.heappop(heap) # шаг: берём минимум
```

```
taken = {start} # дерево растёт от start
```

```
print(f"берём {v} за {w} (из {parent})")`,
```

```
variables: [
 { name: "v", role: "вершина, которую только что д"},
 { name: "u", role: "ещё не взятый сосед v – канди"},
 { name: "w", role: "вес ребра (v, u) – ключ в куче"},
]
```

```

 { name: "n", role: "длина строки s, для которой с
 { name: "i", role: "индекс текущего символа – счи
 { name: "j", role: "длина текущего наилучшего сов
 { name: "π[i]", role: "префикс-функция: длина наи
 ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepDriven: true,
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]
 code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения p
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
 variables: [
 { name: "n", role: "длина строки s", range: "n = 7"
 { name: "i", role: "позиция, для которой считаем l, r"
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]"
 { name: "z[i]", role: "длина наибольшего общего префикса s[0:i] и s[i:r]"
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем поиск в нём
 code: `trie = {"": {}} # бор: вершина → словарь

```

```

half = 1 << (k - 1)
for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
 + half, c + half, k - 1, k - 1]))`,
stepCodeLines: [1, 8],
variables: [
 { name: "r, c", role: "левый верхний угол блока, ",
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг",
 { name: "st2[(r,c,k,k)]", role: "минимум квадратн
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[(r, c, kx, ky)] из вкладки «построение:
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [[0] + [None] * m for _ in range(n + 1)] # рамка =
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][

```

```

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * подсматривает через кнопку-«глаз».
 */
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (/^(for|while|def|class|if\s|elif\s|else|try|except|
 keep.push(line);
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim() === '')
 keep.pop();
 return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, demoId: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n# кнопкой-«глазом»\n`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```



```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 ы рассылать 10 000 писем, он пишет одно письмо в
 счетах сотрудников только тогда, когда соотв
 женное обновление).</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border-rose-500 shadow-md">
 Аналогия 2: Матрешки-коробки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ше, и так далее. Если нам нужно покрасить по
 Внутри всё синее!" на большую коробку. Опус
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Строгое доказательство / Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}</pre>
 </div>
 </details>
 </div>
</div>

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансирует",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вится корнем дерева (VIP-статус). Если ты член
 ка почти не требуется времени!</p>
 </div>

```

Оно поднимет узел, на котором  
ска. За счет сдвига этого листа в корень, д  
/p>

<div>

```
<strong class="text-white block
```

Если агент постоянно переключается между находящимися на разных концах дерева, его можно считать свинг-агентом. Постоянное свинг-переключение превратит

<div>

```
<strong class="text-white block
```

Хотя один поиск может стоить  
емах обладают прекрасным свойством: они не  
по пути. "Палочное" дерево прессуется в сб  
осов.

&lt;/section&gt;`

4.

3

```
id: "prefix-sums-2d",
```

```
title: "5. Двумерная матрица префиксных сумм".
```

```
type: "html",
```

```
description: "Сжатие суммы подматрицы за $O(1)$ ".
```

category: "Алгоритмы матрицы".

content: `

```
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10":
```

```
<div class="flex items-center mb-6 flex-wrap gap-3":
```

`<span class="bg-indigo-600 text-white px-4 py-1`

## class="text-2xl sm:text-3xl font-bold text-violet-600">

&lt;/div&gt;

```

category: "Графы. Пути",
content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Алгоритм Дейкстры
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Представьте себе человека, который не знает, как пользоваться автомобилем.
 <p class="text-xl font-mono text-white">Он знает, что это транспортное средство, но не понимает, как им управлять.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Он знает, что это транспортное средство, но не понимает, как им управлять.
 (нельзя поехать и заработать). Он всегда в состоянии ожидания.
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Он знает, что это транспортное средство, но не понимает, как им управлять.
 дный и не умеет пересматривать уже "зафиксированные" маршруты.
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
</section>`

```

```

 </div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-700/50
border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг <code class="bg-slate-900 text-pink-400">
 шу себе проходить через вершину k?</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересечение
 e> через путь <code class="text-indigo-300">
 Шаг k=2: Разрешаем пересечение

```

```

<ol class="text-sm text-slate-300" style="list-style-type: none;">
 Добавляем фиктивную вершину
 Запускаем от неё медленные обходы «потенциалы»
 Меняем старые веса: новое равенство (старые веса + разность потенциалов) - минимальное значение потенциалов сокращаются).
 Теперь смело запускаем быстрые обходы

</div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 18. Графы. Остовное дерево. Краскал
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Остовное дерево</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">Алгоритм Краскала</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Идея: Построить остовное дерево, добавляя рёбра, не создающие циклов.</p>
 <p class="text-xl font-mono text-white">1. Выбираем рёбра, не создающие циклов, пока не останется n-1 рёбро. (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```

```

 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
</div>
</div>
</div>
</section>`
},
{
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
 </div>
 </div>
 </div>
`

```



```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

♂ Аналогия 1: Поиск без отка

```
</div>
```

```
<p class="text-slate-300 text-sm mb">
```

Мы идём по строке слева направо. строки СЕЙЧАС закончился под моим пальцем?»

Представь, ты ищешь в тексте слова `х`. Тупой алгоритм начнет искать за `</code>`

```
</p>
```

```
</div>
```

```
<!-- Аналогия 2 -->
```

```
<div class="bg-slate-900 p-6 rounded-lg">
```

```
<div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
```

Аналогия 2: LLM стриминг

```
</div>
```

```
<p class="text-slate-300 text-sm mb">
```

Для LLM, которая стримит токены на каждом шаге. Если мы частично совпали с записью какого места этого слова мы можем продолжить

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
<summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<p>Тот же пример: abcbabcd</p>
```

```
<ul class="list-disc pl-5 space-y-2">
```

```
a: Префиксов нет. <code>
```

```
ab: Из начала ("a")
```

```
>
```

```
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «прикидываю
 я начну читать строку отсюда, насколько длин
 Это самый быстрый способ найти i
 >, считаешь Z-функцию, и там, где <code>Z[i]
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>
 <p class="text-slate-300 text-sm mb-4">
 В LLM Z-функция может применяться
 [i]</code> (где <i>i</i> указывает назад на
```

```

 </details>
 </div>
 </div>
 </section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("

```

```

 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb

```

```
(V + E).</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb
гупо, но проверка "знает ли Вася Петю" мгр
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
 </div>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400"

 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный
 </div>
 <p class="text-slate-300 text-sm mb
 Что делает: Жадный алгоритм находит ближайшую
ую вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
```

```
vector<vector<int>>> adj;
```

```
void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}
```

```
<pre class="bg-slate-950 p-3 rounded
```

```
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;
```

```
while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "graph-planar-colors",
```

```
 title: "7. Графы. Планарные. Покраска",
```

```
 type: "html",
```

```
 description: "Формула Эйлера и теорема о 4 красках.",
```

```
 category: "Графы. Теория",
```

```
 content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3"
```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-emerald-300 italic"
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb"
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400

```





```

 <p class="text-slate-300 text-sm mb-4">
 SCC (Билет 10) склеивает
 </p>

 <p class="text-slate-300 text-sm mb-4">
 Точки сочленения (Билет 12)
 Точка сочленения — это вершина графа, удаление которой
 разрушает связность монолита. Вырвешь точку сочленения —
 граф распадется.
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg p-4 border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе, чтобы найти
 <code>strongly connected components</code> (сильно связные компоненты).
 Разворачиваем этот список (то есть берем компоненты в обратном порядке).
 Переворачиваем все ребра в графе.
 Идем по <code>order</code>: для каждой компоненты в обратном порядке
 находим все ребра, ведущие из этой компоненты в другие компоненты.
 Если таких ребер нет, то эта компонента — это и есть SCC.

 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">

```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </p>
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg bor">
 <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
 </p>
 </div>
```

```

 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
}
if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}

```

```

 </div>
 </div>
</details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl border"
 <h4 class="text-lg font-bold text-indigo-400"
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентир
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300"
 <strong class="text-rose-400">Точки
 on> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
 ависимые "конгломераты".
 Как точка сочленения рушит
 Косарайю, свернув каждую SCC в одну супер-в
 (сделаем неориентированным), то любая
 о и есть критическая SCC! Если удалить эту
 . Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 }

```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];
```

```

 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curren

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(

```

```

 };

 useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-rose-400" >
 <div className="bg-slate-800 p-4 border-b border-rose-400" >
 <h3 className="font-bold text-white flex items-center" >
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>
 <div className="flex items-center gap-2 bg-slate-700 p-4" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-2 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 </button>
 </div>
 </div>
 </div>
);
 }
}

```

```
}}}
```

```

{/* Nodes */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited.has
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}

```

```

 </g>
);
 }}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-8 h-1 bg-rose-500 bor
 Мост
 </div>
 </div>
 </div>
 </div>

```



```
import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 useVizStepSync(currentStepIndex, setCurrentStepIndex, setHasCycle);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фуря заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
 updated ? `Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]} ` : ` `
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }
 }
}

```

```

 }, [hasCycle]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-1px border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white rounded-lg">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">Загрузка...</h3>
 <p className="text-sm text-blue-300">Полный исходный код</p>
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border-1px border-slate-800">
 <button onClick={() => setHasCycle(false)} className="p-2 text-blue-600 text-white" : 'text-slate-400'>Отключить</button>
 <button onClick={() => setHasCycle(true)} className="p-2 text-blue-600 text-white" : 'text-slate-400'>Включить</button>
 </div>
 <div>
 <button onClick={() => { setIsPlaying(false); setHasCycle(false); }} className="p-2 text-blue-600 text-white px-3 py-2 rounded-lg">Пауза</button>
 <button onClick={() => setIsPlaying(!isPlaying)} className="p-2 text-blue-600 text-white px-3 py-2 rounded-lg">Воспроизвести</button>
 </div>
 </div>
 </div>
)

```

```

const isNeg = edge.w < 0;
const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

let strokeColor = '#475569';
let marker = 'url(#arrow-bf-normal)';
if (isChecking) { strokeColor = '#f59e0b'; }
else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={isChecking && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} strokeDash={isChecking ? [4, 4] : []} />
 <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Negative Cycle' : 'Cycle'} font-size="11" font-weight="bold" />
 </g>
);
}}}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode === node.id;

 return (
 <g key={node.id} className="transition-ghost">
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? 10 : 5} stroke={isCheckingNode ? '#7dd3fc' : '#64748b'} stroke-width={isCheckingNode ? 2 : 1} />
 <text x={node.x} y={node.y + 5} fill="white" font-size="10" font-weight="bold" text-anchor="middle" />
 <text x={node.x} y={node.y - 28} fill="white" font-size="10" font-weight="bold" text-anchor="middle" />
 <text x={node.x} y={node.y + 35} fill="white" font-size="10" font-weight="bold" text-anchor="middle" />
 </g>
);
})}

```

```

 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
v>
 </div>
);
 }}}
 </div>
 </div>
 </div>

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5
 e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
 }}}
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

---

```
const queue: number[] = [];
const visited = new Set<number>();

// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0).",
});

queue.push(0);
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по",
});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`,
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
```

```

 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);
 useVizStepSync(stepIdx, setStepIdx, STEPS.length - 1)

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length
 }, []));

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

```

```

<div className="bg-slate-800 border border-slate-600 border-radius-50px]">
 <svg className="w-full h-full min-h-[350px]" >
 {/* Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500 ${
 isVisited ? 'stroke-slate-600' : 'stroke-slate-400'
 }`}
 />
)
 })}

 {/* Вершины */}
 {NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#1e293b'; // slate-800
 stroke = '#475569'; // slate-600
 scale = 1;
 }

 return (
 <circle
 key={n.id}
 cx={n.x} cy={n.y}
 fill={fill}
 stroke={stroke}
 r={n.radius * scale}
 className="transition-all duration-500"
 />
)
 })}


```



```

 Шаг {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in grap' },
 { line: 7, code: ' if neighbor not in' },
 { line: 8, code: ' visited.add(ne' },
 { line: 9, code: ' queue.push(nei'
]}.map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
 });
])}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`${qId}-${i}`} className=
 enter justify-center rounded-md shadow-lg sl

```

```
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
 };

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor} shadow-2xl`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}`} >{info.caption}</p>
 </div>
);
}
```

---

ФАЙЛ 34/87: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
```

```

 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);
};

```

```

const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []);

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect(
 },
 [cancelHide]
);

```

```

300 hover:text-white hover:border-emerald-500
>
{saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
{saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF среде"
 className="flex items-center gap-1.5 text-x-small"
 300 hover:text-white hover:border-indigo-500
>
 <Printer className="w-3.5 h-3.5" /> Печать
</button>

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-sm" >
 ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-0" />

 Подчёркнутые термины – интерактивные: наведение
 объяснение на пальцах, мини-анимацию и кнопки

```

```

 <ChapterNav prev={prev} next={next} index={index} />
 </article>

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
 </>
);
};

```

ФАЙЛ 36/87: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Undo, Play, Pause, SkipForward, RefreshCw } from 'lucide-react';

interface CodeLine {
 line: string;
 highlight: string;
}

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: "", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },

```

```

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 }
];

```

```

{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. low[A]=min(low[A],low[B])=1. (A,B) – не мо",
 activeCodeLine: [11,13,16,17]
}

```



```

<button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded
 <Undo className="h-3.5 w-3.5" />
</button>
<button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px]
 dfsAuto ? "bg-rose-600 text-white" : "bg-
 `}>
 {dfsAuto ? <><Pause className="h-3 w-3" />
</button>
<button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 className="p-1.5 hover:bg-slate-700 rounded
 <SkipForward className="h-3.5 w-3.5" />
</button>
<button onClick={() => { setDfsAuto(false); s
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
 {/* Graph visualization */}
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.ke
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && {

```

```

 </g>;
 }}}}
 </svg>
 <div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all duration-300" >
 <div>
 </div>
 </div>
 </div>
 </div>
 </div>

 {/* Code panel + stack */}
 <div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border border-slate-800" >
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400" >
 <div className="p-2 overflow-x-auto" >
 <pre className="text-[10px] font-mono leading-relaxed" >
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl)
 return <div key={i}
 className={`px-1 transition-all duration-300
 ${hl ? "bg-indigo-600/20 text-indigo-400" : ""}
 `}>

 {cl.line || ' '}
 </div>;
 })}
 </pre>
 </div>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
 <div className="text-[10px] font-bold text-slate-400" >
 Стек DFS
 </div>
 </div>
 </div>
 </div>
</div>

```

```
import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
];
```

```

log: ` Дейкстра готов к доставке. Начинаем из ПИ
activeCodeLine: 2,
});

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

```

```

if (!u) break;

```

```

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited }
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
});

```

```

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

```

```

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited }
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

```

```

 if (dist[u] + edge.w < dist[v]) {

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white px-4 py-2 rounded-lg"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py-2 rounded-lg ${isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-slate-700 hover:bg-slate-600'}`}
 >
 {isPlaying ? 'Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (currentStepIndex < steps.length - 1) { setCurrentStepIndex(currentStepIndex + 1); } }}
 disabled={currentStepIndex >= steps.length}
 className="flex items-center gap-1 bg-indigo-600 text-white px-3 py-2 rounded-lg text-sm font-medium transition-colors"
 >
 Шаг
 </button>
 </div>
 </div>
 </div>
);

```

```

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOpt}
 strokeWidth={isChecking ? 5 : isOpt}
 markerEnd={marker}
 className="transition-all duration-1"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f87192'}
 />
 <text
 x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)}
 fill={isChecking ? '#f59e0b' : '#f87192'}
 fontSize="11" fontWeight="bold" text={edge.w}
 />
 </g>
);
 }
}

```

```

{/* Вершины */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-1">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}

```

```

 te-300'}`}>
 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2" >
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep === idx
 return (
 <span key={idx} className={`px-3 py-2
 isCheckingThis ? 'bg-amber-500' : 'bg-white'
 600 text-slate-300'`} >
 {edge.v}
 {edge.v}<span class="font-size-12px"

 an>

);
 })}
 </div>
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 /* Таблица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10" >
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>

```

```

 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line
 return (
 <div key={item.line} className={`py-1 px-4
 ${isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-600'}
 >
 {item.line}
 {item.code}
 </div>
)
 })}
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 38/87: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

```



```

 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
}

if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',

```

```

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 { /* Header & Controls */ }
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 </h3>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-800" >
 N:
 <select>
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950" >
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 />
 </div>
 </div>
 </div>
 </div>
)
}

```

```

 {Array.from({ length: targetN }, (_, i) => i + 1)
 .map(num => {
 const isCached = num in memoState || num <= 2;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center
 ${isCurrent
 ? 'bg-emerald-950 border-emerald-500'
 : isCached
 ? 'bg-slate-900 border-emerald-500/50'
 : 'bg-slate-900/50 border-slate-800'}
 `}
 >
 N
 <span className={`text-xl md:text-2xl font-medium
 ${isCached ? val : '0'}
 `}>{isCached ? val : '0'}
 {num <= 2 &&
 Текущий N: <strong className="text-emerald-500">{currentStep?.n}
 }
 </div>
);
 })
 }
 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-800">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-900/50 border-slate-800 p-2">
 Текущий N: <strong className="text-emerald-500">{currentStep?.n}
 </div>
 </div>
)}
</div>

```

```

 <div className="p-4 bg-slate-900 rounded-xl
 t-slate-300">
 Всего шагов симуляции: {steps.length}
 <span className="text-emerald-400 font-bo
 </div>
 </div>
)}

```

```

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl bord
 <div className="bg-slate-900 px-6 py-3 bord
 <span className="text-xs font-bold font-m
 <span className="text-xs text-emerald-400
 </div>
 <pre className="p-6 font-mono text-sm space
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4
 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate
 }`}
 >
 <span className="w-8 text-xs text-s
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
 </div>
 </div>

```

```

 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);
 };
}

```

```

 className="transition-all duration-300" />
)))
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n.id
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-pointer"
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
 strokeWidth={2.5}
 className="transition-all duration-200" />
 <text x={n.x} y={n.y+4} textAnchor="middle">
 {n.label}</text>
 </g>
)}
)}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500 text-white p-2"
 >Решено: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
 : "bg-slate-900/50 border-slate-700 text-slate-500"}
 >
 {c1Msg}
 </div>
</div>
);
}

```

```

],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты прие
 ля чего нужен алгоритм Беллмана-Форда.'
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки
 options: [
 'Он использует меньше памяти благодаря коммунистиче
 'Он умеет работать с отрицательными весами без ци
 'Он позволяет выполнять шаги слияния колхозов (ко
 'Он был разработан в Токио и поэтому самый быстрый
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая ком
 лелить вычисления на GPU или многопроцессорных кл
},
{
 id: 4,
 question: 'В чем главная суть Динамического програм
 options: [
 'Писать код очень быстро, динамично нажимая на кл
 'Запоминать (кешировать) результаты уже решенных
 'Использовать динамическую типизацию как в Python
 'Постоянно менять требования к проекту в процессе
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
```

```

 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
 });

 if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты</h3>
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов</p>

 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase">Оценка</p>
 <p className="text-5xl font-black text-indigo-500">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
 : score >= 3
 ? ' Отличный результат! Главные концепции усвоены!'
 : ' Стоит повторить главы пособия и еще раз пройти тест.'
 }
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
 }
}

```



```

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full"}
 order-indigo-500 bg-indigo-500 text-white"
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-l"
 {option}
 </div>
 </div>
);
 }
 </div>
 </div>

 {isAnswered && (
 <div className="bg-slate-950 border border-slate-900 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

```

```

const [isPlaying, setIsPlaying] = useState(false);

const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: number } } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' }
];

```

```

 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ' Все пары отработаны. Алгоритм Флойда завершён.'
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 Загрузка...
 </h3>
 </div>
 </div>
 </div>
 </div>
);

```

```

const isVia = (currentStep.i === u && c
let strokeColor = '#475569'; let marker
if (isTarget) { strokeColor = '#10b981'
else if (isVia) { strokeColor = '#818cf8'
return (
 <g key={`-${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x} / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x} / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
});
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8';
 else if (isI) { fill = '#b45309'; stroke = '#b45309';
 else if (isJ) { fill = '#047857'; stroke = '#047857';
 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={isI
sJ ? 4 : 2} className="transition-colors du
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y + 32} fill=
 </text>
 </g>
);
 }}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400

```

```

 /* Матрица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10"
 <div className="flex justify-between items-center"
 <h4 className="text-lg font-bold text-rose-950"
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border"
 <thead><tr className="bg-rose-950/40 text"
 <th className="p-2 border-r border-rose-950"
 {nodeNames.map((n, idx) => <th key={idx}
 -400 font-bold' : ''})`}>{n.split(' ')[0]} {
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b borde"
 <td className={`p-2 bg-rose-950/40`
 tStep.k === i ? 'text-amber-400' : ''})`}>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -1 &&
 tStep.j === j});
 let bg = 'bg-slate-900/40 text-slate-950'
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50'
 else if (isVia) bg = 'bg-indigo-950/60 text-indigo-50'
 else if (currentStep.i === -1 && currentStep.k === -1)
 bg = 'bg-indigo-950/60 text-indigo-50'
 else if (i === j) bg = 'bg-slate-900/40 text-slate-950'
 return (<td key={j} className={`p-2`
 /td>));
 })))
 </tr>
 </tbody>
 </table>
 </div>
 </div>
 </div>

```

```

 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];

const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 }
}

```

```

 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `04
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
}

steps.push({ type: 'backtrack', node: '', desc: `04
 om(vis)] });

return steps;
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera
 useVizStepSync(stepIdx, setStepIdx, steps.length -

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 });
 }
 });
}

```

```

 <line key={i} x1={u.x}
 className={`stroke-
500' : 'stroke-emerald-500'}) : 'stroke-slate
 });
 }}}

```

```

 {/* Nodes */}
 {nodes.map(n => {
 const visited = isVisited(n)
 const current = isCurrent(n)

 let fillColor = "fill-slate
 let strokeColor = "stroke-s
 let textColor = "fill-slate
 let radius = 18;

 if (visited) {
 fillColor = mode === 'd
 strokeColor = mode ===
 textColor = "fill-white
 }

 if (current && step.type !==
 strokeColor = "stroke-a
 textColor = "fill-amber
 radius = 22;
 }
 })

```

```

 return (
 <g key={n.id} className=
 <circle cx={n.x} cy=
 className={`stro
 <text x={n.x} y={n.y}
 className={`tex
 {n.label}
 </text>

```

```

 {/* Ripple for BFS v

```



```

</div>

<div className="flex flex-col sm:flex-row gap-2"
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg p-2 text-slate-400 hover:text-white hover:bg-slate-800"
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={() => setAutoPlay(!autoPlay)}
 justify-center min-w-[80px] ${mode === 'df' ? 'text-slate-400' : 'text-white'}
 >
 {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setSteps(steps.length - 1)}}
 className="p-2 text-slate-400 hover:bg-transparent"
 >
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setSteps(steps.length - 1)}}
 className="p-2 text-slate-400 hover:bg-transparent"
 >
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border border-slate-800 rounded-lg p-2"
 row-hidden">
 <div className="text-[10px] text-slate-400"
 {steps.length}</div>
 <div className="text-sm text-slate-400"
 {step.desc}
 </div>
 </div>
</div>
</div>
);
}

```

```

 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: 'Болит сердце', emoji: '👤', priority: 100 },
 { name: 'Маша (Перелом)', symptom: 'Открытый перелом', emoji: '👤', priority: 80 },
 { name: 'Кот Барсик (Укус)', symptom: 'Укусил шмеля', emoji: '🐈', priority: 60 },
 { name: 'Семён (Температура)', symptom: 'Жар 39.5', emoji: '👤', priority: 40 },
 { name: 'Оля (Порез)', symptom: 'Глубокий порез', emoji: '👤', priority: 20 },
 { name: 'Пётр (Кашель)', symptom: 'Простуда', emoji: '👤', priority: 10 },
];

const INITIAL PATIENTS: Patient[] = (() => {

```

```

const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
const finalName = name || preset.name.split(' ')[0];
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x => x);
// Mark specifically this new patient as 'in' in the heap
const target = nextHeap.find(x => x.priority === priority);
const marked = nextHeap.map(x => x.id === (target ? target.id : null));

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
}, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;

```

```

 addLog(` Успех! Пациент ${topPatient.name} пр
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
}, 650);
}, [busy, heap]));

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}

```

```
</button>
```

```
{/* Добавить кастомного */}
```

```
<form onSubmit={handleCustomInsert} className="flex flex-col items-center gap-4">
```

```
 <input
```

```
 type="text"
```

```
 required
```

```
 value={customName}
```

```
 onChange={e => setCustomName(e.target.value)}
```

```
 placeholder="Имя пациента"
```

```
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
```

```
 />
```

```
 <div className="flex flex-col items-center gap-4">
```

```
 Приоритет
```

```
 <input
```

```
 type="range"
```

```
 min="10"
```

```
 max="99"
```

```
 value={customPriority}
```

```
 onChange={e => setCustomPriority(Number(e.target.value))}
```

```
 />
```

```
 <div className="w-20 accent-emerald-500 cursor-pointer">
```

```
 <div>Взять</div>
```

```
 </div>
```

```
 <button
```

```
 type="submit"
```

```
 disabled={busy || !customName.trim() || heap.length === 0}
```

```
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
```

```
 />
```

```
</form>
```

```
 <div>Взять</div>
```

```
</button>
```

```
</form>
```

```
{/* Забрать самого тяжелого */}
```

```
<button
```

```
 onClick={extractMax}
```

```
 disabled={busy || heap.length === 0}
```

```

 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'ani
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full r
 <div className="bg-slate-950 border
 0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
 <div className="absolute top-4 left-4 text-[10

```

```
 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800" >
 <div className="text-[10px] font-mono text-slate-400" >
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}>
 >
 {idx === 0 ? [
 {entry}
]</div>
 </div>
 </div>
))}
 </div>
 </div>
);
};
```

---

ФАЙЛ 44/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```

 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведение"}
 </div>
 </div>
);
 };
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);
};

```

```

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line

```



```

 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {

```

```

 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];

```

```

 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };

```

```

 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

```

```

 return (
 <Svg caption={step === 0 ? "Граф распался на острова

```

```

 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
))
 </g>
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idle}
 })}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты — огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>

```

```

} > = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idl} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: x под p"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x"}]}
]}
 />
);

```

```

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}}
 </Svg>
);
 };

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}

```

```

 <text x={224} y={62} textAnchor="middle" fontSize=
 <text x={224} y={74} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
 <Edge a={[60, 70]} b={[200, 70]} color={improved
 <text x={130} y={54} textAnchor="middle" fontSize=
 <Node x={60} y={70} r={20} fill={C.active} label=
 <Node x={200} y={70} r={20} fill={improved ? C.do
 <text x={60} y={106} textAnchor="middle" fontSize=
 <text x={200} y={106} textAnchor="middle" fontSiz
 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился н
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} colo

```

```

const chosen = ["A-D", "A-B", "B-C", "D-E"];
const step = useLoop(chosen.length + 1, 800);
const taken = new Set(chosen.slice(0, step));
return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 });
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step
 })}
 </Svg>
);
};

```

```

 <rect x={94} y={100 - i * 26} width={72} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
 <text x={130} y={111 - i * 26} textAnchor="middle" fill={C.dim}>
 </g>
)}}
 <text x={130} y={16} textAnchor="middle" fontSize={14}>
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = ["A", "B"], ["A", "B", "C"], ["B", "C"];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к концу"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
 <text x={53 + i * 54} y={71} textAnchor="middle" fill={C.dim}>
 </text>
 </g>
)}}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);

```

```
export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, Math.min(step - 1, levels.length - 1))}`}>
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} fill={wave?.includes(n.id) ? C.active : C.dim}>
 })}
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={wave?.includes(n.id) ? C.active : C.dim}>
 })}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очередной уровень
 </Svg>
);
};
```

```
export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0], [4, 1], [5, 1], [6, 2], [7, 2], [8, 3], [9, 3], [10, 4], [11, 4], [12, 5], [13, 5], [14, 6], [15, 6], [16, 7], [17, 7], [18, 8], [19, 8], [20, 9], [21, 9], [22, 10], [23, 10], [24, 11], [25, 11], [26, 12], [27, 12], [28, 13], [29, 13], [30, 14], [31, 14], [32, 15], [33, 15], [34, 16], [35, 16], [36, 17], [37, 17], [38, 18], [39, 18], [40, 19], [41, 19], [42, 20], [43, 20], [44, 21], [45, 21], [46, 22], [47, 22], [48, 23], [49, 23], [50, 24], [51, 24], [52, 25], [53, 25], [54, 26], [55, 26], [56, 27], [57, 27], [58, 28], [59, 28], [60, 29], [61, 29], [62, 30], [63, 30], [64, 31], [65, 31], [66, 32], [67, 32], [68, 33], [69, 33], [70, 34], [71, 34], [72, 35], [73, 35], [74, 36], [75, 36], [76, 37], [77, 37], [78, 38], [79, 38], [80, 39], [81, 39], [82, 40], [83, 40], [84, 41], [85, 41], [86, 42], [87, 42], [88, 43], [89, 43], [90, 44], [91, 44], [92, 45], [93, 45], [94, 46], [95, 46], [96, 47], [97, 47], [98, 48], [99, 48], [100, 49], [101, 49], [102, 50], [103, 50], [104, 51], [105, 51], [106, 52], [107, 52], [108, 53], [109, 53], [110, 54], [111, 54], [112, 55], [113, 55], [114, 56], [115, 56], [116, 57], [117, 57], [118, 58], [119, 58], [120, 59], [121, 59], [122, 60], [123, 60], [124, 61], [125, 61], [126, 62], [127, 62], [128, 63], [129, 63], [130, 64], [131, 64], [132, 65], [133, 65], [134, 66], [135, 66], [136, 67], [137, 67], [138, 68], [139, 68], [140, 69], [141, 69], [142, 70], [143, 70], [144, 71], [145, 71], [146, 72], [147, 72], [148, 73], [149, 73], [150, 74], [151, 74], [152, 75], [153, 75], [154, 76], [155, 76], [156, 77], [157, 77], [158, 78], [159, 78], [160, 79], [161, 79], [162, 80], [163, 80], [164, 81], [165, 81], [166, 82], [167, 82], [168, 83], [169, 83], [170, 84], [171, 84], [172, 85], [173, 85], [174, 86], [175, 86], [176, 87], [177, 87], [178, 88], [179, 88], [180, 89], [181, 89], [182, 90], [183, 90], [184, 91], [185, 91], [186, 92], [187, 92], [188, 93], [189, 93], [190, 94], [191, 94], [192, 95], [193, 95], [194, 96], [195, 96], [196, 97], [197, 97], [198, 98], [199, 98], [200, 99], [201, 99], [202, 100], [203, 100], [204, 101], [205, 101], [206, 102], [207, 102], [208, 103], [209, 103], [210, 104], [211, 104], [212, 105], [213, 105], [214, 106], [215, 106], [216, 107], [217, 107], [218, 108], [219, 108], [220, 109], [221, 109], [222, 110], [223, 110], [224, 111], [225, 111], [226, 112], [227, 112], [228, 113], [229, 113], [230, 114], [231, 114], [232, 115], [233, 115], [234, 116], [235, 116], [236, 117], [237, 117], [238, 118], [239, 118], [240, 119], [241, 119], [242, 120], [243, 120], [244, 121], [245, 121], [246, 122], [247, 122], [248, 123], [249, 123], [250, 124], [251, 124], [252, 125], [253, 125], [254, 126], [255, 126], [256, 127], [257, 127], [258, 128], [259, 128], [260, 129], [261, 129], [262, 130], [263, 130], [264, 131], [265, 131], [266, 132], [267, 132], [268, 133], [269, 133], [270, 134], [271, 134], [272, 135], [273, 135], [274, 136], [275, 136], [276, 137], [277, 137], [278, 138], [279, 138], [280, 139], [281, 139], [282, 140], [283, 140], [284, 141], [285, 141], [286, 142], [287, 142], [288, 143], [289, 143], [290, 144], [291, 144], [292, 145], [293, 145], [294, 146], [295, 146], [296, 147], [297, 147], [298, 148], [299, 148], [300, 149], [301, 149], [302, 150], [303, 150], [304, 151], [305, 151], [306, 152], [307, 152], [308, 153], [309, 153], [310, 154], [311, 154], [312, 155], [313, 155], [314, 156], [315, 156], [316, 157], [317, 157], [318, 158], [319, 158], [320, 159], [321, 159], [322, 160], [323, 160], [324, 161], [325, 161], [326, 162], [327, 162], [328, 163], [329, 163], [330, 164], [331, 164], [332, 165], [333, 165], [334, 166], [335, 166], [336, 167], [337, 167], [338, 168], [339, 168], [340, 169], [341, 169], [342, 170], [343, 170], [344, 171], [345, 171], [346, 172], [347, 172], [348, 173], [349, 173], [350, 174], [351, 174], [352, 175], [353, 175], [354, 176], [355, 176], [356, 177], [357, 177], [358, 178], [359, 178], [360, 179], [361, 179], [362, 180], [363, 180], [364, 181], [365, 181], [366, 182], [367, 182], [368, 183], [369, 183], [370, 184], [371, 184], [372, 185], [373, 185], [374, 186], [375, 186], [376, 187], [377, 187], [378, 188], [379, 188], [380, 189], [381, 189], [382, 190], [383, 190], [384, 191], [385, 191], [386, 192], [387, 192], [388, 193], [389, 193], [390, 194], [391, 194], [392, 195], [393, 195], [394, 196], [395, 196], [396, 197], [397, 197], [398, 198], [399, 198], [400, 199], [401, 199], [402, 200], [403, 200], [404, 201], [405, 201], [406, 202], [407, 202], [408, 203], [409, 203], [410, 204], [411, 204], [412, 205], [413, 205], [414, 206], [415, 206], [416, 207], [417, 207], [418, 208], [419, 208], [420, 209], [421, 209], [422, 210], [423, 210], [424, 211], [425, 211], [426, 212], [427, 212], [428, 213], [429, 213], [430, 214], [431, 214], [432, 215], [433, 215], [434, 216], [435, 216], [436, 217], [437, 217], [438, 218], [439, 218], [440, 219], [441, 219], [442, 220], [443, 220], [444, 221], [445, 221], [446, 222], [447, 222], [448, 223], [449, 223], [450, 224], [451, 224], [452, 225], [453, 225], [454, 226], [455, 226], [456, 227], [457, 227], [458, 228], [459, 228], [460, 229], [461, 229], [462, 230], [463, 230], [464, 231], [465, 231], [466, 232], [467, 232], [468, 233], [469, 233], [470, 234], [471, 234], [472, 235], [473, 235], [474, 236], [475, 236], [476, 237], [477, 237], [478, 238], [479, 238], [480, 239], [481, 239], [482, 240], [483, 240], [484, 241], [485, 241], [486, 242], [487, 242], [488, 243], [489, 243], [490, 244], [491, 244], [492, 245], [493, 245], [494, 246], [495, 246], [496, 247], [497, 247], [498, 248], [499, 248], [500, 249], [501, 249], [502, 250], [503, 250], [504, 251], [505, 251], [506, 252], [507, 252], [508, 253], [509, 253], [510, 254], [511, 254], [512, 255], [513, 255], [514, 256], [515, 256], [516, 257], [517, 257], [518, 258], [519, 258], [520, 259], [521, 259], [522, 260], [523, 260], [524, 261], [525, 261], [526, 262], [527, 262], [528, 263], [529, 263], [530, 264], [531, 264], [532, 265], [533, 265], [534, 266], [535, 266], [536, 267], [537, 267], [538, 268], [539, 268], [540, 269], [541, 269], [542, 270], [543, 270], [544, 271], [545, 271], [546, 272], [547, 272], [548, 273], [549, 273], [550, 274], [551, 274], [552, 275], [553, 275], [554, 276], [555, 276], [556, 277], [557, 277], [558, 278], [559, 278], [560, 279], [561, 279], [562, 280], [563, 280], [564, 281], [565, 281], [566, 282], [567, 282], [568, 283], [569, 283], [570, 284], [571, 284], [572, 285], [573, 285], [574, 286], [575, 286], [576, 287], [577, 287], [578, 288], [579, 288], [580, 289], [581, 289], [582, 290], [583, 290], [584, 291], [585, 291], [586, 292], [587, 292], [588, 293], [589, 293], [590, 294], [591, 294], [592, 295], [593, 295], [594, 296], [595, 296], [596, 297], [597, 297], [598, 298], [599, 298], [600, 299], [601, 299], [602, 300], [603, 300], [604, 301], [605, 301], [606, 302], [607, 302], [608, 303], [609, 303], [610, 304], [611, 304], [612, 305], [613, 305], [614, 306], [615, 306], [616, 307], [617, 307], [618, 308], [619, 308], [620, 309], [621, 309], [622, 310], [623, 310], [624, 311], [625, 311], [626, 312], [627, 312], [628, 313], [629, 313], [630, 314], [631, 314], [632, 315], [633, 315], [634, 316], [635, 316], [636, 317], [637, 317], [638, 318], [639, 318], [640, 319], [641, 319], [642, 320], [643, 320], [644, 321], [645, 321], [646, 322], [647, 322], [648, 323], [649, 323], [650, 324], [651, 324], [652, 325], [653, 325], [654, 326], [655, 326], [656, 327], [657, 327], [658, 328], [659, 328], [660, 329], [661, 329], [662, 330], [663, 330], [664, 331], [665, 331], [666, 332], [667, 332], [668, 333], [669, 333], [670, 334], [671, 334], [672, 335], [673, 335], [674, 336], [675, 336], [676, 337], [677, 337], [678, 338], [679, 338], [680, 339], [681, 339], [682, 340], [683, 340], [684, 341], [685, 341], [686, 342], [687, 342], [688, 343], [689, 343], [690, 344], [691, 344], [692, 345], [693, 345], [694, 346], [695, 346], [696, 347], [697, 347], [698, 348], [699, 348], [700, 349], [701, 349], [702, 350], [703, 350], [704, 351], [705, 351], [706, 352], [707, 352], [708, 353], [709, 353], [710, 354], [711, 354], [712, 355], [713, 355], [714, 356], [715, 356], [716, 357], [717, 357], [718, 358], [719, 358], [720, 359], [721, 359], [722, 360], [723, 360], [724, 361], [725, 361], [726, 362], [727, 362], [728, 363], [729, 363], [730, 364], [731, 364], [732, 365], [733, 365], [734, 366], [735, 366], [736, 367], [737, 367], [738, 368], [739, 368], [740, 369], [741, 369], [742, 370], [743, 370], [744, 371], [745, 371], [746, 372], [747, 372], [748, 373], [749, 373], [750, 374], [751, 374], [752, 375], [753, 375], [754, 376], [755, 376], [756, 377], [757, 377], [758, 378], [759, 378], [760, 379], [761, 379], [762, 380], [763, 380], [764, 381], [765, 381], [766, 382], [767, 382], [768, 383], [769, 383], [770, 384], [771, 384], [772, 385], [773, 385], [774, 386], [775, 386], [776, 387], [777, 387], [778, 388], [779, 388], [780, 389], [781, 389], [782, 390], [783, 390], [784, 391], [785, 391], [786, 392], [787, 392], [788, 393], [789, 393], [790, 394], [791, 394], [792, 395], [793, 395], [794, 396], [795, 396], [796, 397], [797, 397], [798, 398], [799, 398], [800, 399], [801, 399], [802, 400], [803, 400], [804, 401], [805, 401], [806, 402], [807, 402], [808, 403], [809, 403], [810, 404], [811, 404], [812, 405], [813, 405], [814, 406], [815, 406], [816, 407], [817, 407], [818, 408], [819, 408], [820, 409], [821, 409], [822, 410], [823, 410], [824, 411], [825, 411], [826, 412], [827, 412], [828, 413], [829, 413], [830, 414], [831, 414], [832, 415], [833, 415], [834, 416], [835, 416], [836, 417], [837, 417], [838, 418], [839, 418], [840, 419], [841, 419], [842, 420], [843, 420], [844, 421], [845, 421], [846, 422], [847, 422], [848, 423], [849, 423], [850, 424], [851, 424], [852, 425], [853, 425], [854, 426], [855, 426], [856, 427], [857, 427], [858, 428], [859, 428], [860, 429], [861, 429], [862, 430], [863, 430], [864, 431], [865, 431], [866, 432], [867, 432], [868, 433], [869, 433], [870, 434], [871, 434], [872, 435], [873, 435], [874, 436], [875, 436], [876, 437], [877, 437], [878, 438], [879, 438], [880, 439], [881, 439], [882, 440], [883, 440], [884, 441], [885, 441], [886, 442], [887, 442], [888, 443], [889, 443], [890, 444], [891, 444], [892, 445], [893, 445], [894, 446], [895, 446], [896, 447], [897, 447], [898, 448], [899, 448], [900, 449], [901, 449], [902, 450], [903, 450], [904, 451], [905, 451], [906, 452], [907, 452], [908, 453], [909, 453], [910, 454], [911, 454], [912, 455], [913, 455], [914, 456], [915, 456], [916, 457], [917, 457], [918, 458], [919, 458], [920, 459], [921, 459], [922, 460], [923, 460], [924, 461], [925, 461], [926, 462], [927, 462], [928, 463], [929, 463], [930, 464], [931, 464], [932, 465], [933, 465], [934, 466], [935, 466], [936, 467], [937, 467], [938, 468], [939, 468], [940, 469], [941, 469], [942, 470], [943, 470], [944, 471], [945, 471], [946, 472], [947, 472], [948, 473], [949, 473], [950, 474], [951, 474], [952, 475], [953, 475], [954, 476], [955, 476], [956, 477], [957, 477], [958, 478], [959, 478], [960, 479], [961, 479], [962, 480], [963, 480], [964, 481], [965, 481], [966, 482], [967, 482], [968, 483], [969, 483], [970, 484], [971, 484], [972, 485], [973, 485], [974, 486], [975, 486], [976, 487], [977, 487], [978, 488], [979, 488], [980, 489], [981, 489], [982, 490], [983, 490], [984, 491], [985, 491], [986, 492], [987, 492], [988, 493], [989, 493], [990, 494], [991, 494], [992, 495], [993, 495], [994, 496], [995, 496], [996, 497], [997, 497], [998, 498], [999, 498], [1000, 499], [1001, 499], [1002, 500], [1003, 500], [1004, 501], [1005, 501], [1006, 502], [1007, 502], [1008, 503], [1009, 503], [1010, 504], [1011, 504], [1012, 505], [1013, 505], [1014, 506], [1015, 506], [1016, 507], [1017, 507], [1018, 508], [1019, 508], [1020, 509], [1021, 509], [1022, 510], [1023, 510], [1024, 511], [1025, 511], [1026, 512], [1027, 512], [1028, 513], [1029, 513], [1030, 514], [1031, 514], [1032, 515], [1033, 515], [1034, 516], [1035, 516], [1036, 517], [1037, 517], [1038, 518], [1039, 518], [1040, 519], [1041, 519], [1042, 520], [1043, 520], [1044, 521], [1045, 521], [1046, 522], [1047, 522], [1048, 523], [1049, 523], [1050, 524], [1051, 524], [1052, 525], [1053, 525], [1054, 526], [1055, 526], [1056, 527], [1057, 527], [1058, 528], [1059, 528], [1060, 529], [1061, 529], [1062, 530], [1063, 530], [1064, 531], [1065, 531], [1066, 532], [1067, 532], [1068, 533], [1069, 533], [1070, 534], [1071, 534], [1072, 535], [1073, 535], [1074, 536], [1075, 536], [1076, 537], [1077, 537], [1078, 538], [1079, 538], [1080, 539], [1081, 539], [1082, 540], [1083, 540], [1084, 541], [1085, 541], [1086, 542], [1087, 542], [1088, 543], [1089, 543], [1090, 544], [1091, 544], [1092, 545], [1093, 545], [1094, 546], [1095, 546], [1096, 547], [1097, 547], [1098, 548], [1099, 548], [1100, 549], [1101, 549], [1102, 550], [1103, 550], [1104, 551], [1105, 551], [1106, 552], [1107, 552], [1108, 553], [1109, 553], [1110, 554], [1111, 554], [1112, 555], [1113, 555], [1114, 556], [1115, 556], [1116, 557], [1117, 557], [1118, 558], [1119, 558], [1120, 559], [1121, 559], [1122, 560], [1123, 560], [1124, 561], [1125, 561], [1126, 562], [1127, 562], [1128, 563], [1129, 563], [1130, 564], [1131, 564], [1132, 565], [1133, 565], [1134, 566], [1135, 566], [1136, 567], [1137, 567], [1138, 568], [1139, 568], [1140, 569], [1141, 569], [1142, 570], [1143, 570], [1144, 571], [1145, 571], [1146, 572], [1147, 572], [1148, 573], [1149, 573], [1150, 574], [1151, 574], [1152, 575], [1153, 575], [1154, 576], [1155, 576], [1156, 577], [1157, 577], [1158, 578], [1159, 578], [1160, 579], [1161, 579], [1162, 580], [1163, 580], [1164, 581], [1165, 581], [1166, 582], [1167, 582], [1168, 583], [1169, 583], [1170, 584], [1171, 584], [1172, 585], [1173, 585], [1174, 586], [1175, 586], [1176, 587], [1177, 587], [1178, 588], [1179, 588], [1180, 589], [1181, 589], [1182, 590], [1183, 590], [1184, 591], [1185, 591], [1186, 592], [1187, 592], [1188, 593], [1189, 593], [1190, 594], [1191, 594], [1192, 595], [1193, 595], [1194, 596], [1195, 596], [1196, 597], [1197, 597], [1198, 598], [1199, 598], [1200, 599], [1201, 599], [1202, 600], [1203, 6
```



```

<Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -
 о префиксные суммы P`} >
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i <= r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`} >
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={i < len ? C.dim : C.idle} />
 <text x={25 + i * 30} y={29} textAnchor="middle">{a[i]}</text>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (

```

```
};
```

```
export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.idle}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{len}</text>
 </g>
)}}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};
```

```
export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
```

```

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};

```

```

const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
};
window.addEventListener("keydown", onKey);
document.body.style.overflow = "hidden";
return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
};
}, [vizId, onClose]));

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end" >
 <div className="absolute inset-0 bg-slate-950/80" >
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4" >
 <div className="min-w-0 flex-1" >
 <h3 className="font-bold text-white text-sm" >
 {viz.hint} && <p className="text-[11px] text-white" >
 {viz.description}
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain" >
 <Component />
 </div>
 </div>
 </div>
 </div>,

```

```

 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left: left });
 }, [anchor]);

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(

```

```
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
</div>,
document.body
);
};
```

---

ФАЙЛ 51/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync } from '../data/vizStepBus';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP);

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
```

```

 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-emerald-500";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-emerald-500";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-slate-dim)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

```

```

);
case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соединяем ее с остальными вершинами.
 Это наша база для расчета потенциалов.
 </div>
);
case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.

 Запускаем медленный, но мощный алгоритм поиска кратчайших путей от фиктивной вершины S.
 ...
 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес: $w_{ij} + h(v_i) - h(v_j)$.
 <div className="text-center font-monospace">...
 Это повышает или понижает вес ребра.
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.

 Старый вес: ...
 Вычисляем: $-2 + 0 - (-2) = 0$.
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.

 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) = 0$.
 </div>
);

```



```

 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 10" style="position: absolute; top: 10px; left: 10px; color: #FFA500; font-size: 20px; transform: rotate(45deg);">
 start-reverse">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 10" style="position: absolute; top: 10px; left: 10px; color: #FF69B4; font-size: 20px; transform: rotate(45deg);">
 tart-reverse">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 10" style="position: absolute; top: 10px; left: 10px; color: #3CB371; font-size: 20px; transform: rotate(45deg);">
 o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 </defs>

```

```

 { /* Edges */ }
 { edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 return null;
 }

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 x2={v.x} y2={v.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(e)}
 />

```

```

 <rect x="-1"
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
 }}
 </g>
)
 }}}
 </svg>
 </div>

```

```

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-
 {getDesc()}
 </div>

```

```

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border-1
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3} />
 </button>
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border-1
-slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size={24} />
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bold text-white
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}

```

```

}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);

```

```

 -1,
 });

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 initialEdges.forEach((e) => adj[e.u].push(e.v));

 // DFS 1: post-order list
 const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
 false,
 -1,
);
 };

 // Run DFS1 on all unvisited
 for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
 }

```

```

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} комп
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {

```



```

const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 />

```

```

 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && {
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 }}
 </g>
);
}}
</svg>

{step.transposed && {
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
}}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4

```



```

 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>

```

```

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([

```

```
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
}
```

```
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

 // --- PRIM STEPS ---
 const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)', 'F-H (9)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8', '4-9']),
 setLogs(prev => [{

```

```

setVertices(prev => prev.map(v => v.id === 0 ? {
setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
}, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) - цикл',
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},

```

```

 description: 'Плесень из центра Токио (Е) тянет',
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
```





```

 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between two villages)
 () => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}.
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ...e, status: 'included'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мосты',
 description: 'Теперь Красный и Синий колхозы одобрят мосты A-D с весом 5. Остальные мосты B-E(6) и C-F(9) будут отклонены.',
 type: 'warning'
 }, ...prev]);
 },
 // Step 4: Concurrently merge into single Kolkhoz
 () => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'black' })));
 setEdges(prev => prev.map(e => {
 ...e, status: e.status === 'included' || e.id === '0-3' ? 'included' : 'not_included'
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский колхоз'
 }]);
 }
}

```

```

 description: 'Представь, что все 9 вершин графа
 type: 'info',
 },
 });
} else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Hear) и захватываем соседей!',
 type: 'info',
 },
]);
} else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
}
};

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
 };

```

```

 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 }
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

```

```

-center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 {activeAlgo === 'kruskal' ? (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
 </>
) : activeAlgo === 'prim' ? (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-1 bg-sky-600">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
 </>
) : (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
 </div>
 </>
)
)
 }
}

```

```

 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -l
 }}
 </div>
)}}
</div>

```

```

{stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'})}>
 MST построено с помощью {activeAlgo
 </p>
 <p className="text-slate-300 text-xs mt

```

```

 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCirc
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)))
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-40
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal
 className={"px-3 py-1.5 rounded text-xs f

```

```

 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
ожении:</p>
 <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Code className="w-3.5 h-3.5 text-ind
 </p>
 <pre className="text-xs text-emerald-400
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </pre>
 </div>
 </div>
 </div>
})

```

```

while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
return mst`}

```

```

</pre>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

})

```

```

{activeCheatTab === 'boruvka' && {

```

```

 <div className="grid grid-cols-1 lg:grid-cols-2" >

```

```

 <div className="lg:col-span-1 space-y-4">

```

```

 <div className="bg-slate-950 p-4 rounded-3xl">

```

```

 <p className="text-slate-400 text-xs font-medium">

```

```

 <Clock className="w-3.5 h-3.5 text-rose-500" />

```

```

 </p>

```

```

 <p className="text-2xl font-black text-white">

```

```

 <p className="text-slate-400 text-xs mt-1">

```

```

 </div>

```

```

 </div>

```

```

 <div className="bg-slate-950 p-4 rounded-3xl">

```

```

 <p className="text-slate-400 text-xs font-medium">

```

```

 <Award className="w-3.5 h-3.5 text-rose-500" />

```

```

 </p>

```

```

 <p className="text-2xl font-black text-white">

```

```

 <p className="text-slate-400 text-xs mt-1">

```

```

 </div>

```

```

 <div className="bg-slate-950 p-4 rounded-3xl">

```

```

 <p className="text-slate-400 text-xs font-medium">

```

```

 <Award className="w-3.5 h-3.5 text-rose-500" />

```



ФАЙЛ 54/87: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
```

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, []);

 return (
 <>
```

```
 />
 </div>
 </div>
 </div>
 })
</>
});
};
```

---

ФАЙЛ 56/87: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 }
```

```

 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 страницах)"
 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }}
 >
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
 >
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"

```

```

<div className="grid grid-cols-1 md:grid-cols-2 g
 {/* K5 */}
 <div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 }
 }
 }

```

```

 };
 function findPt(id:number) { return pts.f[id]; }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143","\u04144"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2" style="background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; z-index: 20">

```

```

 Square,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { emitVizStep, onVizDemo, vizHitsAtTrace, vizSte
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Перейти к странице пособия (посмотреть визуализац
 onOpenGuide: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) =>
 setStderr: (options: { batched: (message: string) =>
 setStdin: (options: { stdin: () => string | number |
};

```

```
{
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая",
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
 es)\n',
 },
],
},
{
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j — как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик — как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"Итерация {i}\")\ni += 1",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i — позиция, x — элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
```



```

 },
],
},
];

/** Переживает сккрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string };

export function PythonCompiler({ chapterId, chapterTitle }: { chapterId: string; chapterTitle: string }) {
 /** Активная вкладка демонстрации страницы (у sparse-редактора)
 const [demoId, setDemoId] = useState<string | null>(null);
 const sync = useMemo(() => getPageSync(chapterId, demoId));
 const syncVarBases = useMemo(() => sync.variables.map((v) => v.bases));
 const syncVarSet = useMemo(() => new Set(syncVarBases));
 /** Полный эталонный код – подсматривается «глазом», не редактируется
 const template = useMemo(() => buildSyncTemplate(chapterId, sync));
 /** Дефолт редактора: шапка + только инициализированный код
 const initTemplate = useMemo(() => buildInitTemplate(chapterId));
 /** Помеченные строки эталона по содержимому: каждое слово в строке
 const markedContents = useMemo(() => {
 const marks = sync.stepCodeLines;
 if (!marks || marks.length === 0) return undefined;
 const lines = template.split("\n");
 return new Set(marks.map((l) => (lines[l + 2] ?? "")));
 }, [sync, template]);
 /** Режим «подсмотреть эталон»: показывает полный код
 const [showRef, setShowRef] = useState(false);

 const [code, setCode] = useState(() => {
 if (!savedEditor) return initTemplate;
 if (savedEditor.code === savedEditor.template || savedEditor.code === "")
 return savedEditor.code;
 return savedEditor.code;
 });
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

```

```

 setDebug(null); // трасса ссылается на строки старого
 setShowRef(false);
 if (code === prev.base || code === prev.full || code === prev.partial) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setDebug(null);
 setShowRef(false);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(showRef ? template : code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – буфер переполнен");
 }
}, [code, showRef, template]);

/* — Пошаговый отладчик —————

/** Редактор – в точности эталон (для подписи chip).
const lockedToViz = code === template;
/** Пользователь дописал тело сам, перепечатав помощь
const contentMatched = useMemo(() => {
 if (!markedContents || markedContents.size === 0) return false;
 const markedContent = markedContents.get(0);
 const markedContentCode = markedContent.code;
 const markedContentTemplate = markedContent.template;
 return code === markedContentCode || code === markedContentTemplate;
}, [markedContents]);

```



```

 stepBy(-1);
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
};
// capture-фаза: чтобы глобальный обработчик навигации
window.addEventListener("keydown", onKey, true);
return () => window.removeEventListener("keydown", onKey);
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[debug.idx] : undefined };
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current) return;
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком порядке
const debugChanged = curDebug?.step ? changedVars(curDebug?.step) : undefined;
const debugOrdered = curDebug?.step ? orderVars(curDebug?.step) : undefined;
/** Сколько помеченных строк кода уже выполнено к текущему шагу */
const curHits =
 debug && curDebug?.step && markedContents
 ? vizHitsAtTrace(debug.result.steps, debug.idx, debug.markedContents)
 : 0;

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({ snippet: string }) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
 const pos = ta.selectionStart;
 const posEnd = ta.selectionEnd;
 return current.slice(0, pos) + snippet + current.slice(posEnd, current.length);
 });
}, [editorRef]);

```

```

<Tooltip side="bottom" content="Вернуть коммен
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Закрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip
 side="bottom"
 content={
 showRef
 ? "Скрыть эталонный код и вернуться к с
 : "Подсмотреть эталонный код целиком (т
 }
 >
 <button
 type="button"
 onClick={() => setShowRef((v) => !v)}
 className={`p-1.5 rounded-lg transition-co
 hite hover:bg-slate-800`}
 aria-label={showRef ? "Скрыть эталонный к
 >
 {showRef ? <EyeOff className="w-4 h-4" />
 </button>
 </Tooltip>

```

```

 <HelpCircle className="w-3.5 h-3.5" /> Подс
 </button>
 <div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Редактор и визуализация смотрят на"
 : "На выбранной странице нет интерактивн
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex item
 hasViz ? "text-emerald-400" : "text-slate-500"
 }`
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

 </Tooltip>
 </div>
</div>

```

```

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-w-700 text-slate-300 hover:text-white hover:background-color-slate-800">
 <Tooltip side="bottom" content="Открыть эскиз">
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center gap-2"
 >
 {chapterTitle}
 </button>
 </Tooltip>
 </div>
 </div>
) : null}

```

```

 {v.name}
 </code>
 </Tooltip>
)))
 {sync.stepNote && (
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3
 <span className="truncate max-w-5

 </Tooltip>
)}
</div>
)}
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map((gItem) => (
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map((s) => (
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-s
 {s.label}
 </button>
 </Tooltip>

```

```

 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопро
 <button type="button" onClick={() => {
-md text-slate-400 hover:text-white hover:b
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопро
 <button type="button" onClick={() => {
sult.steps.length - 1} className="p-1 round
tion-colors">
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="В конец трассы">
 <button type="button" onClick={() => {
tep || debug.idx >= debug.result.steps.leng
0 disabled:opacity-40 transition-colors">
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 {stepCapable && markedContents && (

)}
 </>
) : (
 "шагов нет – программа не выполнила н
)}

 {debug.result.truncated && (
 <Tooltip content={`Трасса обрезана: лим
шие данные.`}>

```



```

 <Tooltip content="Выход из отладчика (Esc
 <button type="button" onClick={stopDebug}
00/30 transition-colors">
 <Square className="w-3.5 h-3.5" />
 </button>
 </Tooltip>
</div>

```

```

{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-[300px]">
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-4
 active ? "bg-emerald-500/15 text-emerald-300" : ""
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-gray-400"
 }`}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 })}
</div>

```

```

{/* панель переменных – «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden">
 <div className="flex items-center gap-1.5">
 <Variable className="w-3 h-3 text-indigo-300" />
 </div>

```



```
 {runtimeReady && <CheckCircle2 className="w-6 h-6 text-slate-600" />
 {outputOpen}
 </button>
 {outputOpen && (
 <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-slate-200">
 </div>
 </aside>
);
}
```

---

ФАЙЛ 59/87: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-purple-500', bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-indigo-500', bubbleText: 'Матана!' },
];
```

```

 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c), 450);
 }, [queue, isServing]);

 // DEQUEUE: Выдать суп первому (голова / Head, FIFO)
 const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eating' } : c));

 setTimeout(() => {

```

```

 <p className="text-xs text-slate-300 mt-1 lead">
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата – только строгий п
 </p>
 </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-400 to-slate-500 opacity-40 disabled:cursor-not-allowed text-white
 rounded-xl transition-all cursor-pointer"
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-400 to-slate-500 opacity-40 disabled:cursor-not-allowed text-white
 rounded-xl transition-all cursor-pointer"
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

```

```

 { /* СПИСОК ОЧЕРЕДИ */ }
 <div className="flex-1 flex items-end gap-4" >
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-4" >

 <p className="text-xs font-bold font-sans">Очередь пуста</p>
 <p className="text-[10px]">Все сыты</p>
 </div>
) : (
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
 return (
 <div
 key={customer.id}
 className={`flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `>
 <div>
 { /* Облачко мыслей */ }
 <div className={`absolute -top-16 left-1/2 -translate-x-1/2
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border-t' : ''}
 `}>
 <div className="font-bold text-slate-200">
 {customer.bubbleText}
 <div className="absolute -bottom-16 right-1/2
 it rotate-45"></div>
 </div>
 </div>
 { /* Персонаж */ }
 <div className={`relative w-14 h-14 rounded-2xl border-2

```

```

</div>

<div className="flex-1 flex flex-col justify-
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-f
 {servedHistory.map((item, idx) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 round
0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
)}}
 </div>
)}
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slat
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-indigo-5

```

```

const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и буль!');
const [isTalking, setIsTalking] = useState<boolean>(false);
const [isBlinking, setIsBlinking] = useState<boolean>(false);

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false), 1000);
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {

```



```

 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : leftX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД)
 S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем построить БПД');
 return;
 }
 const updated = words.filter(word => word !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

```

```

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.char})`;

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] === undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return {

```

```

>
 { /* Тело лосося */ }
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85 Z" fill="#e67e22" />

 { /* Хвост */ }
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-rotate"
 />

 { /* Плавники */ }
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#1abc9c" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#1abc9c" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-rotate"
 />

 { /* Глаз (с морганием) */ }
 <circle cx="145" cy="85" r="12" fill="#fff" />
 { isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" />
) : (

 <circle cx="148" cy="85" r="6" fill="#000" />
 <circle cx="151" cy="82" r="2" fill="#fff" />
) }
 </>
 </>

 { /* Рот (анимация разговора) */ }
 { isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q 165 110 155 115 168 118" stroke="#000" />
) : (
 <path d="M 165 110 Q 155 115 168 118" stroke="#000" />
) }
 </>

```

```

<div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустит
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузит
 </button>
)}
</div>
</div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево и Суффиксные (fail) ссылки
 </h5>

```

```

 stroke={isCurrent ? '#60a5fa' : 'rg
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x) / 2 - 10}
 y={({parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
</g>
);
 }}}

 {/* Draw Fail Links */}
 {simulationStep > 0 && Object.values(node
 if (node.id === 0 || node.fail === 0) r
n draw all! But maybe wait, we can just draw
 // Wait, all nodes get fail link. Let's
tep >= 1 and computed it
 // To avoid too many lines, let's draw
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !t

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rg
 strokeWidth={isCurrent ? 2 : 1.5}

```

```

 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
}}
</svg>
);
}}())
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}

```

```

<h5 className="text-white font-bold mb-3 flex

 Таблица переходов и fail-

 <span className="text-xs text-slate-400 font
 Всего вершин: {Object.keys(nodes).length}

</h5>

```

```

<div className="max-h-[220px] overflow-y-auto
 <table className="w-full text-left border-c
 <thead>
 <tr className="border-b border-slate-700
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(n

 return (
 <tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold t
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }`}
 >

```

```

 #{node.term_link}

))
 </td>
 <td className="py-2.5 px-3 text-slate-600">
 {Object.keys(node.children).length === 0 ? 'Ø' : Object.entries(node.children)
 .map([c, id] => `${c}→#${id}`)
 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

 Ø

) : (

 {node.words.join(', ')}

)}
 </td>
</tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```



```

const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 lightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] =`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень:`,
 Highlight: [0, n - 1] });
return steps;

```

```

const n = arr.length;
// Build iterative tree: leaves at [n..2n-1]
const tree: Record<number, number> = {};
for (let i = 0; i < n; i++) tree[n + i] = arr[i];
for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

const steps: Step[] = [];
let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR] });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r} нечётный → r--`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR] });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = (l + r) / 2`, codeLine: 7, treeState: tree, highlightNodes: [l, r], arrayHighlight: [qL, qR], result: res });
 l = (l + r) / 2;
 }
}

```

```

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " } return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-900' } }[color];

```

```

 </div>
)}
 </div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 /* Array strip */
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900
 <span className="text-[7px] text-slate-50
 <span className="text-[11px] font-bold fo
 </div>
);
 })}
 </div>

 /* Tree */
 <div className="flex-1 p-2 overflow-x-auto bg-sla
 <div className="scale-[0.85] origin-top min-w-m
 </div>

 /* Code */
 <pre className="px-3 py-2 text-[9px] md:text-[10px]

```

```

 <button onClick={() => { setPlaying(false); s
 .5 text-slate-400 hover:text-white hover:bg
 button>
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12
 const [customInput, setCustomInput] = useState('5, 8,
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();

```

```

 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-500 text-white font-medium rounded-md focus:outline-none focus:ring-indigo-500 focus:ring-offset-2">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (
 A[{i}]
))}
 или
 </div>
 </form>

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-slate-500">
 <label className="text-[10px] font-bold uppercase">Запрос</label>
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 от
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-500 focus:outline-none focus:ring-indigo-500 focus:ring-offset-2"
 />
 </div>
 <div className="flex items-center gap-1">
 по
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-500 focus:outline-none focus:ring-indigo-500 focus:ring-offset-2"
 />
 </div>
 </div>
 </div>

```

```

 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
 })

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2" >
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху` }
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу` }
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b" >
 <h4 className="text-base font-bold text-white">
 Почему разбиение на узлы происходит именно так для N={arr.length}?
 </h4>
 <p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-950 p-1">

```





```

placeholder="Поиск по темам.."
className="w-full bg-slate-800/70 border border-indigo-500 focus:outline-none focus:border-indigo-500"
/>
{query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-indigo-500"
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
</div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm text-slate-500">
 Нет результатов
 </p>
 &&
 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase text-slate-500">
 {category}
 </div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 className={`w-full text-left px-3 py-2 ${
 isSelected ? "bg-indigo-600/15 border-indigo-500" : "bg-slate-800/40 border-transparent"
 }`}
 >
 {chapter.title}
 </button>
)
 })}
 </div>
 </div>
)}
 &&

```

ФАЙЛ 63/87: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

-----

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```
export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |
```

```
 // Stack state
```

```
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]));
```

```
 const [popMessage, setPopMessage] = useState<string |
```

```
 // Queue state
```

```

 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage("Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🤖' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(`Встал в конец очереди: ${newPerson.name}`);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет новых посетителей");
 }
 const person = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(`Служил: ${person.name} (иконка: ${person.icon})`);
};

```

```

 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`    Выбран самый "тяжелый" кандидат: ${best.text}`);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-slate-200 to-slate-500">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md ${
 activeTab === 'stack' ? 'bg-slate-800 text-white' : 'bg-slate-700 text-slate-300'
 }`}
 >
 Стек (LIFO)
 </button>
 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md ${
 activeTab === 'queue' ? 'bg-slate-800 text-white' : 'bg-slate-700 text-slate-300'
 }`}
 >
 Очередь (FIFO)
 </button>
 <button
 onClick={() => setActiveTab('priority')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md ${
 activeTab === 'priority' ? 'bg-slate-800 text-white' : 'bg-slate-700 text-slate-300'
 }`}
 >
 Приоритетная очередь
 </button>
 </div>
 </div>
);

```

```

onClick={() => setActiveTab('heap')}
className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 te
 : 'bg-slate-800/60 border-slate-700/60 te
}`}
>
<div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite

```

```

 </div>
) : {
 <div className="w-full max-w-sm flex flex-direction-column" >
 <div className="absolute top-0 text-xs font-mono" >
 full flex items-center gap-1" >
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 animate-pulse" />
 </div>

 {/* Reversed map so top of stack is visible */}
 {stack.slice().reverse().map(({plate, index}) => {
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-center justify-between p-2
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-102'
 : 'bg-slate-900/90 border-slate-800'}
 `} >
 <div>
 <div className="flex items-center" >
 {plate.name}

 {plate.name}

 </div>
 <div>

 {isTop ? 'Готово к помывке' : ''}

 </div>
 </div>
 </div>
);
 })}
 </div>
)}
)}
</div>
})

```

```

 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
 } : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">

 <span className="text-xs font-bold font
 </div>

 <div className="text-indigo-500 flex it
 <ArrowRight className="w-6 h-6 animat
 </div>
 }
 }

```

```

 })
 <div className="mt-6 text-xs text-slate-500">
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
 </div>
 </div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />

```



```

 ointer h-10"
 >
 <Plus className="w-4 h-4" /> Добавить в
 </button>
</div>
</form>

{heapMessage && {
 <div className="bg-emerald-950/60 border border-1
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rounded
 {heapMessage}
 </div>
 }}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2xl"
 >
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : (
 <div className="w-full max-w-2xl space-y-4">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 100).toFixed(1);
 return (
 <div
 key={item.id}
 className={`flex items-center justify-between

```

```
 </div>
 </div>
);
 }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500" >
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```

---

ФАЙЛ 64/87: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
 /** Открыть тренажёр в модальном окне. */
 onOpen: (vizId: string) => void;
 /** Перейти к теме, к которой привязан тренажёр. */
 onGoChapter: (chapterId: string) => void;
}
```

```
/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
```

```

 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
</div>

{items.length === 0 && <p className="text-slate-5
<div className="grid gap-3 sm:grid-cols-2 xl:grid
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 borde
 >
 <h3 className="font-bold text-white text-sm
 {entry.hint && <p className="text-[12px] te
 {!entry.hint && <div className="flex-1" />}
 <div className="flex items-center gap-2 mt-
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 te
 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Закрыт
 </button>
 {chapterTitle && (
 <button
 onClick={() => onGoChapter(id)}
 title={chapterTitle}

```

```
const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
 },
 {
 name: "Смешанный",
```

```

 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
}

/* ----- раскладка -----

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

```

```

 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonical));

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.length === 1 }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border" style={styles.container}>
 <div className="flex items-start justify-between" style={styles.header}>

```

```

 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 })}

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(n.key) ? "#475569" : "white"}
 stroke={isTarget ? "none" : "#475569"}
 stroke-width={isTarget ? 0 : 1.8}
 />
 </g>
);
 })}
 </div>
</script>

```

```

 </button>

 поворотов: {moves.length} • минимум: {minimal}

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40" >
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 >
 <div className="flex items-center gap-2 font-mono" >
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} шагов`
 : `Узел ${target} в корне за ${moves.length} шагов`
 }
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

)
)
 }

)}
 </div>
)}

```



```

interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелу. */
 moved?: NodeId[];
 ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C: [160, 112],
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g: [255, 40],
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел сглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит узел p",
 focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,

```

```
const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 — 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 — 40",
```

```

return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 /* призраки: где узел стоял до поворота */
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
 });
 });
);
}

{edges.map(([a, b], i) => {

```

```

 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
 }}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].p;
 const duration = useMemo(() => Math.round(current.ms - prevPos), [current.ms, prevPos]);
 const last = idx === total - 1;

 useEffect(() => {
 setFrame(0);
 }, [caseIdx]);
}

```

```

 >
 {current.kind === "aim" ? "прицел" : current.title}

 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading-5">
 </div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>
 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map(({t, i}) => {
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-[10px]
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : ""
 }`}
 >
 {t}

))}

 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

 {/* индикатор автопрокрутки */}
 <div className="h-1 w-full bg-slate-800 rounded-full">
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 >

```

```

 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-3
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-500"
 }`}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto" >
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (

)
)}
 </div>
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не
 </p>
</div>
);

```

```

 x.left = y.right;
 y.right = x;
 return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi
 // Values: 10, 20, 30, 40, 50, 60
 const createInitialTree = (): SplayNode => {
 let r: SplayNode | null = null;
 const initialKeys = [40, 20, 50, 10, 30, 60];
 initialKeys.forEach((key) => {
 r = insertBST(r, key);
 });
 return r!;
 };
};

```



```

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем
);
}

// Now let's implement standard Splay logic recursi
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right
 if (node.left.right) {
 node.left = leftRotate(node.left);
 steps.push(`Компонент Zig-Zag: левый поворо
 }
 }
 }
}

```

```

 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
 };

```

```

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree, val);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

```

```

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

```

```

const resetTree = () => {
 setTree(createInitialTree());
 setHighlightedNode(null);
 setLog(["Дерево возвращено в исходное состояние."]);
};

```

```

 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

```

```

const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 data-title={`Кликните, чтобы выполнить Splay(${
 >
 <circle
 cx={node.x}
 cy={node.y}
 r="18"
 fill={isHighlighted ? "#4f46e5" : "#1e293b"}
 stroke={isHighlighted ? "#818cf8" : "#64748b"}
 strokeWidth={isHighlighted ? "3" : "2"}

```

```

<svg
 className="w-full h-full max-w-[600px] max-h-[320px]"
 viewBox="0 0 600 320"
>
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
</svg>

{/* Quick interactive controls */}
<div className="absolute bottom-4 left-4 right-4" >
 <form onSubmit={handleFind} className="flex" >
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 className="w-24 bg-slate-950 text-white"
 />
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo-700"
 >
 Поиск / Splay
 </button>
 <button
 type="button"
 onClick={(e) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (!isNaN(val)) {
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStep(newTree, val);
 if (newRoot) setTree(newRoot);
 setLog([`Вставили ${val}`], ...steps);
 setInputValue("");
 }
 }}
 >
 Вставить
 </button>
 </form>
</div>

```

```

 </div>

 <div className="border-t border-slate-800 pt-1">
 <p>

 у самого корня. Делаем 1 вращение.

 </p>
 <p>

 оба левые или оба правые. Вращаем деда, по

 </p>
 <p>

 колено. Вращаем узел в разные стороны.

 </p>
 </div>
 </div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-slate-800">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нитья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3">
 <div className="bg-slate-900/60 p-4 rounded-l">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>
 <p className="text-slate-400 leading-relaxed">
 Допустим, мы ищем{" "}
 <code className="text-indigo-400 font-s">
 его нет. Алгоритм спустится к листу, на
 неудачей (например,{" "}
 <code className="text-indigo-400">[20]<

```

```
 <p className="font-bold text-slate-300 mb-3">3. Амортизация $O(\log n)$ </p>
 <p className="text-slate-400 leading-relaxed">Парадокс: один запрос за{" "}
 <code className="text-emerald-400">O(n)</code> всю структуру вдвое более плоской за счет
 при двойных вращениях {
 <code className="text-emerald-400 font-mono">O(n)</code>
 }. Дорогая операция инвестирует в дешевые операции. Поэтому амортизированное время
 <strong className="text-white"> $O(\log n)$
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emerald-500">Каждое "растяжение" компенсируется "сжатием"</div>
 </div>
</div>
);
};
```

---

ФАЙЛ 68/87: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
```

```
interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
```

```

 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.id}`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id));
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.');
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);
 setDirtyStack(prev => prev.slice(0, -1));
 setShake(true);
 setTimeout(() => setShake(false), 400);
}, [dirtyStack, isWashing]);

const pop = () => popPlate();

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Сбросить (RESET)
 </button>
 </div>
 </div>
)

```



```

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map((plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
 ${plate.color}
 ${plate.animState === 'in' ? 'ani
 ${plate.animState === 'washing' ?
 ${isTop ? 'ring-4 ring-blue-500/5
 `}
 >

```

```

 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-100
 >
 {plate.name}
 {plate.id}
 </div>
))}
 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4" />
 <div className="text-[10px] font-mono text-slate-400" />
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`
 >
 {idx === 0 ? ЛОГ ДЕЙСТВИЙ :
 {entry}

 </div>
))}
 </div>
</div>
);
};

```

---

```

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }
 }

```

```

useVizStepSync(stepIdx, setStepIdx, steps.length -
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>

```

```

 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-aqua-500";
 bgColor = step.match === "z" ? "bg-aqua-500" : "bg-emerald-900";
 }
 }

 return (
 <div key={index} className={
 /* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r ? "border-l" : ""}
 <div className="absolute left-0 top-0 bottom-0 right-0" style={{
 border: "1px solid " + borderColor,
 background: bgColor,
 }}
 {mode === "z" && step.r <= index && index <= step.l ? "border-r" : ""}
 <div className="absolute left-0 top-0 bottom-0 right-0" style={{
 border: "1px solid " + borderColor,
 background: bgColor,
 }}

 /* Index */
 <span className={`text-${step.zTarget === index ? "white" : "black"} font-mono font-bold`}
 style={{ position: "absolute", left: "50%", top: "50%", transform: "translate(-50%, -50%)" }}
 />

 /* Char Box */
 <div className={`w-8 h-8 text-${step.zTarget === index ? "white" : "black"} font-mono text-lg transition-colors`}
 style={{ position: "absolute", left: "50%", top: "50%", transform: "translate(-50%, -50%)" }}
 />

 /* Value array box */
 <div className="text-[1.2em] font-mono font-bold" style={{ position: "absolute", left: "50%", top: "50%", transform: "translate(-50%, -50%)" }}
 style={{ background: "black", color: "white", padding: "5px 10px", border: "1px solid black" }}
 />
)
)
}

```

```

 })

 { /* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }}}}
</div>
</div>

<div className="flex flex-col sm:flex-row g
 { /* Controls */
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled:
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled:
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={reset} className="
border-slate-800">
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

```

```

 if (s[i] == s[j]) j++;
 pi[i] = j;
 }` : `int l = 0, r = 0;
 for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
 }` }
 </pre>
 </div>
</div>
);
}

```

ФАЙЛ 70/87: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";

interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры

```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];
```



```

);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in topological order)
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
 u,
);
 }
 }
 };

 fullyProcessed.add(u);
 dfsStack.pop();
 topoList.push(u); // prepend behavior: we write in reverse order
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {

```

```

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center

```

```

const u = dagNodes.find((n) => n.id === e.u)
const v = dagNodes.find((n) => n.id === e.v)

const isActive =
 (step.currentNode === edge.u || step.currentNode === edge.v) ||
 step.visited.has(edge.v);

return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow-processed"})`}
 className="transition-all duration-300"
 />
);
}}}

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {

```

```

 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>
 </div>
</div>

```

```
 disabled={currentStepIdx >= steps.length}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
```

```

 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы – куча сама держит лучшую
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.
 Component: SegmentTreeVisualizer,
 },
 "sparse-table": {

```

```

 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
 },
 "bellman-ford": {
 title: "Форд–Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
 },
 floyd: {
 title: "Флойд–Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: () => {} },
 "mst-kruskal": { title: "Краскал и DSU", Component: () => {} },
 "mst-prima": { title: "Прим", Component: () => {} },
 "mst-boruvka": { title: "Борувка", Component: () => {} },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () => {} },
 "string-z-func": { title: "Z-функция", Component: () => {} },
 "aho-corasick": {
 title: "Ахо–Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 </div>
),
 },

```

```

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, d: 100, r: 100,
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, d: 100, r: 100,
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, d: 100, r: 100,
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, d: 100, r: 100,
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, d: 100, r: 100,
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d: 100, r: 100,
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, d: 100, r: 100,
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, d: 100, r: 100,
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d: 100, r: 100,
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, d: 100, r: 100,
 = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('');
 const [currentIndex, setCurrentIndex] = useState<number>(0);

```



```

let curr = currentNode;
let logMsg = `[Символ '${char}'] `;

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
} else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Логика завершена.`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не достиг. Переходим по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 curr = nextNode;
 }
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;

```

```

title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: null,
},
{
targetNodeId: 7,
title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: null,
},
{
targetNodeId: 2,
title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плавает в ROOT и ищет ветку по "E".
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'E',
checkingBranchesFrom: 0,
},
{
targetNodeId: 5,
title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
codeLine: 3,
scoutPos: 0,

```

```

 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С...
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро...
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
],
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700">
 /* Шапка и переключатель режимов */
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Анимация водопада: Полное обучение автомата
 </h4>

```

```

<div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди BFS (начиная с
 арг 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font
 : 'bg-slate-800 text-slate-400
 }` }
 >
 {step
 {idx === trainStep && <span classNa
 </button>
)}}
 </div>
 </div>
 </div>

```

```

<div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button

```

```

 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold m
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>
 <h5 className="text-white font-bold mb-2
 » Текст для сканирования

```

```

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium transition-colors
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 `}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
 </div>
 </div>

 <div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 ✂ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <div className={`p-1.5 rounded flex items-center justify-between
 border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && Инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
 border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie[curr].get(char))
 {activeSearchCodeLine === 2 && Выход по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
 border-blue-400 text-blue-200 font-bold`} >
 3. curr = trie[curr].get(char)

```

```

<h5 className="text-white font-bold text-base" style="text-align: center;">
 Ветвящийся синий водопад (Борис)
</h5>
<div className="flex flex-wrap gap-4 text-xs" style="text-align: center;">

</div>
</div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar" style="text-align: center;">
 <svg viewBox="0 0 800 520" className="w-full h-full" style="display: inline-block; vertical-align: middle;">

 {/* Горизонтальные линии и подписи уровней */}
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
 { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
 { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
 { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="140" height="24" style="fill: #0000FF; opacity: 0.4;"
 />
 </g>
 })}
 </div>

```

```

 fill={isTrainingExamined ? (isMatchBranch ? 'red' : 'green') : 'gray'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПУНКТИРНЫЕ ВОДОПАДНЫЕ ССЫЛКИ */}
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x - toNode.x) / 2})`}>
 <rect x="-15" y="-12" width="30" height="24"
 stroke="black" strokeWidth="2" />
 <text x="0" y="5" fontSize="14" fontStyle="italic">
 {isMatchBranch ? ' ' : ' ' }
 </text>
 </g>
)}
 </g>
 </svg>
);
});
}}}

```

```

{/* Рендер fail-ссылки (пунктирные водопадные ссылки) */}
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.failId];

```

```

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработали этот шаг
 const stepIdxForNode = TRAINING_STEPS_INFO[node.id].stepIdx;
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

```

```

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
 // Если активна fail-ссылка

```



```

 <circle cx={node.x} cy={node.y} r="
 }}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r="
 }}

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24'
 fill={isCurrent ? '#2563eb' : isTar
 stroke={isCurrent ? '#ffffff' : isTa
 strokeWidth={isCurrent || isTargetCh
 className="transition-all duration-
 />

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ?
 fontSize={isCurrent || isTargetChil
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
 {isTargetChild && (
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
)}

```

```

 </g>

 </svg>
 </div>
 </div>

 /* Список найденных совпадений (только в режиме поиска)
 {activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded" >
 <h5 className="text-white font-bold mb-3 text-sm" >
 Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic" >
 Пока ничего не поймали. Запустите шаги симуляции
 </p>
) : (
 <div className="flex flex-wrap gap-2" >
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]" >
 <CheckCircle2 className="w-4 h-4 text-emerald-500" />
 {match.word}

 Индекс {match.index}

 </div>
))}
 </div>
)}
 </div>
)}
</div>
);
};

```

```

const [mode, setMode] = useState<"1d" | "2d">("1d");
const chapterId = useContext(VizChapterContext);

// Вкладка 2D = своё демо (свой скелет): компилятор п
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d"
}, [chapterId, mode]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border
 <div className="flex gap-2 mb-4 overflow-x-auto no
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`}
 >
 {label}
 </button>
)))
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[
 []

```

```

const inCover = covered && i >= covered.f
const inRange = i >= range.l && i <= range.r
const isDrivenAdd = driven !== null && driven < 0
return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref" })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange({r} => (i < range.r ? i : range.r))}
 className={` ${cellBase} cursor-pointer ${
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2 ring-black"
 : inCover
 ? "bg-indigo-500 text-white ring-2 ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === 0;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const masked = driven !== null && i > driven;
 const isDrivenCur = driven !== null && i === driven;
 return (
 <div
 key={i}

```

```

 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 $P[\{hover.i\}] = \{pref[hover.i]\}$
 {" "}
 – это сумма всего, что набежало от начала

 $a[0..\{hover.i - 1\}] = \{A1.slice(0, hove$

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 $a[\{hover.i\}] = \{A1[hover.i]\}$
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 $sum(a[\{range.l\}..\{range.r\}]) = P[\{range.r + 1$
 \{pref[range
```

```

 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
 };

 return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
)
 </div>
)
 </div>
 </div>
 </div>
 </div>
 </div>

 {/* Матрица префиксных сумм */}

```

```

 </div>
 </div>

 <div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j] +$
 {" "}
 – сумма всего прямоугольника от левого верхнего угла
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
 </div>

 <div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A + C + {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
 </div>
);
 };
};

```

ФАЙЛ 75/87: src/components/visualizers/SparseTableViz.tsx  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 724 | РАЗМЕР: 1.5 КБ

```

import React, { useContext, useEffect, useState } from 'react';
import { emitVizDemo, useVizStepSync, VizChapterContext } from 'viz';

```

```

 Array(4).fill(0)
)
}
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_build2"&> "1d";
 const chapterId = useContext(VizChapterContext);

 // Каждая вкладка = своё демо с собственным скелетом:
 useEffect(() => {

```



```

 </div>
);
};

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for each j
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 useVizStepSync(step, setStep, maxStep);
 const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

 return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 </div>
 </div>
 </div>
);
};

```

```

{hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{ " "}

 min({arrID.slice(covered.from, covered.to)})

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-100">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 { /* Headers */ }
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">

```

```

// Let's re-eval exact source:
// We are asking if THIS cell [i][j]
if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true
 if (currentStep.i + (1 << (currentStep.j - 1)) === i)
 isSrc2 = true;
}
}

return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors"
) : (
 <motion.div
 onMouseEnter={() => isVisible = true}
 onMouseLeave={() => setHover(false)}
 onClick={() => isVisible && setHover(true)}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors
 hover && hover.i === i && hover.i === j
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"}
 >
)}
 >

```

```

 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return {
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(

 ST[{c.i}][{c.j - 1}]

 ,

 ST[{c.i + half}][{c.j - 1}]

) → min(
 {stl

 {stlD[c.i + half][c.j - 1]}

) ={" "}

 {stlD[c.i][c.j]}

 </p>
);
 })()
) : {
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
 </div>
</div>
);
};

```

```

<div className="flex flex-col items-center gap-1
 scrollbar">
 {[...Array(k + 1)].map((_, i) => k - i).map((i) => {
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === k;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate-500">
 <div className={`flex flex-col items-center gap-1`} >
 <h4 className={`font-bold mb-3 flex items-center`} >
 Матрица блоков {stepL}x{stepL}
 <span className={`text-[10px] font-mono font-weight-normal
 border-indigo-800/40 text-indigo-300` : `bg-slate-800/40
 shadow-[0_0_25px_rgba(99,102,241,0.4)]`} >
 </h4>
 <div className="max-w-full overflow-hidden" >
 <div className={`grid gap-[2px] p-[5px] border-indigo-800/40 shadow-[0_0_25px_rgba(99,102,241,0.4)]`} >
 <div className="grid grid-cols-8" >
 {Array.from({length: stepSize * stepSize}, (_, i) => {
 const r = Math.floor(idx / stepSize);
 const c = idx % stepSize;

 let highlightClass = isCurr ? `bg-indigo-800/50 cursor-pointer` : `bg-slate-800/50 text-slate-500`;
 let zIndex = `z-0`;

 const isActive = !!activeCell && (activeCell.r === r && activeCell.c === c);

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo-800/50 cursor-pointer`;
 zIndex = `z-10`;
 }
 }
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
 }
 </React.Fragment>
);
 })}
 </div>

```

```

 </div>
);
 }}}}
 </div>
</div>
</div>
</React.Fragment>
)
}}}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-4" >
 <div className="bg-slate-900 p-4 sm:p-6 rounded" >
 <h3 className="text-lg font-bold text-slate-300" >
 {k === 0 ? (
 <div className="text-slate-400 text-sm leading-6" >
 <p className="mb-4" >При
 ы имеют размер
 <div className="bg-[#0a0f1e] rounded-xl p-4" >
 // Ба
 ST
 </div>
 <p className="mt-6 italic text-indigo-400" >
 <Play size={14} className="fill-indigo-400" />
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-fade-in" >
 <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto" >
 // Те
 int1);

 // Кв
 ST

 e="text-amber-300" >1], <span

```

```

 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
 </div>
 <div className="flex iter
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex iter
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}())}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
 }}
</div>
}}
</div>

```

```

y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>

```

```

)
 }}}

```

```

<div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

```

```

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-e
 shadow-[0_0_10px_#f43f5e]" style={{ top: `
 lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
 <div className="absolute pointer-e
 shadow-[0_0_10px_#3b82f6]" style={{ top: `
 width: `calc(`${(Ly / 8) * 100}% - 16px)`, l
 <div className="absolute pointer-e
 ld-400 shadow-[0_0_10px_#10b981]" style={{
 8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
 00 shadow-[0_0_10px_#f59e0b]" style={{ top:
 00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
 </div>

```

```

 <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen

```



```

 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<

В запроса)

r1][

во, чтобы правый край уперся в c2)<b

r1][

рх, чтобы нижний край уперся в r2)<b

nded">r2 - Lx + 1][

и влево от c2)

">r2 - Lx + 1][
 {'}'}));
 </div>

 <div className="w-full flex justify-center"
 <div className="flex flex-col items-center"
 <h4 className="text-slate-400 font-bold">
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]

```

```
]]">
 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
 <span className="text-slate-400 fon

 <span className="text-blue-400 m
 n>
 <span className="text-emerald-40
 span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>
 </div>
 </div>
)
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

---

ФАЙЛ 76/87: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

---

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
```

```

 document.querySelectorAll('[id^="theme-"]')
 .forEach(btn => btn.classList.remove('outline', 'outline-1'));

 if (theme === 'dark') document.getElementById('dark')?.classList.add('text-white transition-colors');
 if (theme === 'light') document.getElementById('light')?.classList.add('text-white transition-colors');
 if (theme === 'terminal') document.getElementById('terminal')?.classList.add('text-white transition-colors');

 document.getElementById('btn-theme-dark')?.classList.add('text-white transition-colors');
 document.getElementById('btn-theme-light')?.classList.add('text-white transition-colors');
 document.getElementById('btn-theme-notebook')?.classList.add('text-white transition-colors');
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlighting
 document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 100);
 }
 } else {
 mobileDrawer.classList.add('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 }
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay?.addEventListener('click', toggleDrawer);
 });

```

```

 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href
 link.classList.add('bg-
 } else {
 link.classList.remove('l
 }
 });
 }
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>

```

ФАЙЛ 77/87: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1000

```

<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.

```

```
 display: none;
 }

 mjx-container {
 max-width: 100% !important;
 }
 mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::aft

 .uno-equation .text-2xl { font-size: 1.2rem
 .uno-equation.gap-4 { gap: 0.5rem !important

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !importan
 px; }
 }
 .uno-inner {
```

```
body.theme-notebook {
 background-color: #fcf8eb !important; /* light beige */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(
 45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 {
 color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 {
 border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
```

```

</marker>
<marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
</marker>
<marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1"
</marker>
<marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b1"
</marker>
<marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
</marker>
<marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
</marker>
<marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro

```

```

<!-- Левое меню (Оглавление) -->
<aside id="mobile-drawer" class="fixed inset-y-0
 nslate-x-full transition-transform duration-300
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-y-auto
 lg:rounded-2xl lg:border lg:border-slate-700
 <div class="flex justify-between items-center
 <h3 class="font-black text-transparent bg-clip-text
 ider text-sm">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="text-slate-500
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
 <path stroke-linecap="round stroke-width="2" d="M6 18
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">
 <div>
 <details class="group" open>
 <summary class="list-none cursor-pointer text-slate-500

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 1.1
 1.2
 1.3
 1.4
 </div>
 </div>
 </details>
 <div>
 <details class="group">
 <summary class="list-none cursor-pointer text-slate-500

 2. Группы

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 2.1
 2.2
 2.3
 2.4
 </div>
 </div>
 </div>
 </nav>
 </div>
 </div>
 </div>
 </div>

```



```
</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-500">
 5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text-fuchsia-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-500">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-500">

 </div>
</details>

<details class="group">
```



```


 14. Кратность корня. Корень

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 deg)
 К
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многочлены

 </summary>
 <div class="pl-6 mt-2 space-y-2">
```

```

<script>
 document.querySelectorAll('details.'
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
});

// Scroll spy logic
document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;

 document.querySelectorAll('.grou
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });

 const observer = new Intersection
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting
 visibleId = entry.t

 // Highlight active
 document.querySelec
 a.style.fontWei

```

```
modal.style.backgroundColor = 'white';
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за проблем с некоторыми веб-студиями в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
```

```

 }
 btnContainer.appendChild(closeB
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal
}

```

```

function downloadMD() {
 let clone = document.querySelec

 // Smart markdown parsing based

 // Smart markdown parsing based
 clone.querySelectorAll('h2').fo
 clone.querySelectorAll('h3').fo
 clone.querySelectorAll('.analog
 clone.querySelectorAll('li').fo
 clone.querySelectorAll('svg').f
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
 });
 clone.querySelectorAll('.img-cap

 let text = clone.innerText;
 text = text.replace(/\n{3,}/g,

 if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
 }
 const blob = new Blob([text], {
 const url = URL.createObjectURL
 const link = document.createEle
 link.href = url;
 link.download = 'vyisshaya alge

```

```

body.bg-white aside > d
body.bg-white [class*="
nt; }
body.bg-white [class*="
nt; }
body.bg-white [class*="
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc

```

```

 <button onclick="downloadWord()"
ont-bold flex items-center justify-center gap-2
 Word
 </button>
 <a href="algebra.pdf" target="_blank"
-sm font-bold flex items-center justify-center gap-2
 PDF

 <button onclick="downloadMD()"
-b bold flex items-center justify-center gap-2
 Markdown
 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center gap-2
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
 </div>
 </div>
 </div>
</div>
</aside>

```



```
</header>
```

```
<!-- ===== WTF CONTAINER =====
```

```
<div id="wtf-container" class="hidden">
```

```
 <h2 class="text-3xl font-black text-cen
```

```
 <div class="overflow-x-auto w-full pb-4
```

```
 <div class="grid grid-cols-3 gap-4
```

```
 <!-- Column headers -->
```

```
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
```

```
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
```

```
 <div class="bg-cyan-900/40 bord
сные числа</div>
```

```
 <!-- Row 1: База -->
```

```
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
```

```
 <div class="text-xs text-yellow
```

```
 <div class="font-bold text-slat
```

```

```

```
 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
```

```
 <div class="text-xs text-orange
```

```
 <div class="font-bold text-slat
```

```

```

```
 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
```

```
 <div class="text-xs text-cyan-5
```

```
 <div class="font-bold text-slat
```

```

```

```
 <!-- Row 2: Базовые операции / Евкл
```

```
 <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
```

```
 <div class="text-xs text-yellow
```

```
 <div class="font-bold text-slat
```

```

-4 border-cyan-500 hover:bg-slate-700 transition
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

<!-- Row 5: Корни / Уравнения -->
<div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

-4 border-cyan-500 hover:bg-slate-700 transition
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1">

```

```
 <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500"
 <div class="font-bold text-slate-900"

 </div>
</div>
```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="mb-6"
 <div class="flex items-center mb-6"
 <div class="bg-red-500/20 p-3 rounded-r-
 <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
 <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-l-4"
 </div>
```

```
 <div class="prose prose-invert max-w-4xl"
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
 <h2 class="text-2xl font-bold border-l-4"
(Меметика)</h2>
```

```
 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6"
 <h3 class="text-xl text-blue-400"
 <p class="text-sm text-slate-400"
 <ul class="list-disc pl-5"
 <span class="text-white"
k$).
 <span class="text-white"

```

валось.</li>

Просто перепиши формулу с буквой  $k$ .</li>

усок  $k$ , замени его на жареного карася из

</div>

<!-- Двойная делимость -->

<div class="bg-slate-800/80 p-6

<h3 class="text-xl text-ora

<p class="text-sm text-slate

<ul class="list-disc pl-5 sp

вверх по уравнениям).</li>

рху вниз, показывая, что  $\Delta r_i$

х. Твой  $d$  больше.</li>

</ul>

</div>

<h2 class="text-2xl font-bold b  
оритмы экзекуции)</h2>

<div class="grid grid-cols-1 md

<div class="bg-slate-800 p-

<h4 class="text-red-400

<p class="text-sm text-

<p class="text-xs font-

со следующим. Если в конце 0 – пациент мерт

</div>

<div class="bg-slate-800 p-

<h4 class="text-red-400

<p class="text-sm text-

<p class="text-xs font-

```


 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 x двух, потом последних.

 <code class="text-x">
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение – есть ноль. Если есть умноже
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается
 <code class="text-x">
 </div>

 </div>
 </section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

```
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid`
```

```
// <details> в файле лучше раскрыть: иначе код и разб
host.querySelectorAll("details").forEach((d) => d.set

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
 </div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
 <h2>Словарик терминов из этой темы</h2>
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
 ${items}
 </section>`;
}

function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
```

```
 setTimeout(() => URL.revokeObjectURL(url), 2000);
 }

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^\\s*(\\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-": ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter): Promise<string> {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]): Promise<string> {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}>${c.title}</a>`)
 .join("\\n");

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 allTerms.push(t);
 bodies.push(body);
 });
 });

 const allTermsText = allTerms.join("\\n");
 const allBodiesText = bodies.join("\\n");

 const html = wrap({
 title: "Универсальное пособие",
 subtitle: "Полное содержание",
 css,
 body: `<div>${allTermsText}</div>${allBodiesText}</div>`,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "Универсальное пособие.html");
}
```



```

* node scripts/audit-coverage.mjs # отчёт
* node scripts/audit-coverage.mjs --md # markd
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\\s{2}[a-zA-Z0-9_]+/g)].map(m => m[0]));

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 };
});

```

```

 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.joi
}
```

ФАЙЛ 81/87: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
```

```

console.log("Не встретились:", GLOSSARY.filter((g) => !
```

```

ФАЙЛ 82/87: scripts/export/build-pdf.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ
```

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, } from

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],

```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+$/, "Конфигурация проекта"],
```

```

 };
 walk(ROOT);
}

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

/** Достаём заголовки глав из данных пособия (без запуска) */
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("src/"))) {
 const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
 return na - nb;
 });
}

```

```

 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");

```

```
/** DPI раstra. Можно переопределить: EXPORT_DENSITY=
density: Number(process.env.EXPORT_DENSITY ?? 150),
/** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
monochrome: process.env.EXPORT_GRAYSCALE !== "1",
size: "A4",
font: "DejaVu-Sans-Mono",
fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
pointsize: 8,
/** Максимум символов в строке (далее – жёсткий пере
cols: 138,
/** Строк содержимого на странице (+2 служебные: шапка
rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore"
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
```

```

 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}

function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 im.call("convert", txtFile, pngFile);
 });
}

```



```

 if (missing.length) throw new Error(`Не отрендерены с

 const bytes = jobs.reduce((s, j) => s + fs.statSync(j
 console.log("[2/3] txt -> png");
 console.log(`      страниц: ${total}`);
 console.log(` выход: ${path.relative(ROOT, PAG
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});
```

=====

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

-----

ФАЙЛ 86/87: [.github/workflows/deploy-pages.yml](#)  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

-----

name: Deploy app to GitHub Pages

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧ
(у бота-интеграции нет admin-прав на них – API отвеча
#
1. Settings → Pages → Build and deployment → Source
Если там «Deploy from a branch» (legacy) – шаг D
2. Settings → Environments → github-pages → Deploym
добавьте паттерн «arena/**» и ветку «main» (или
иначе деплой из новой рабочей ветки отклоняется
```

```
on:
 push:
 branches:
 - main
```

- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
  path: ./dist

deploy:

name: Publish to GitHub Pages  
needs: build  
runs-on: ubuntu-latest  
environment:  
  name: github-pages  
  url: \${ steps.deployment.outputs.page\_url }

steps:

- # Ранняя и понятная ошибка вместо невнятного «Fail»
- # когда в репозитории Pages не переключён на сбор

- name: Preflight – Pages собирается через GitHub

env:

GH\_TOKEN: \${ secrets.GITHUB\_TOKEN }

run: |

bt=\$(gh api "repos/\${ github.repository }/pages/build\_type")

echo "Pages build\_type: \${bt}"

if [ "\${bt}" != "workflow" ]; then

  echo "::error title=Pages не настроен::Settings не настроены. Проверьте этот запуск. См. комментарий в начале деплоя"

  exit 1

fi

- name: Deploy  
id: deployment  
uses: actions/deploy-pages@v5

---

ФАЙЛ 87/87: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

---

name: Rebuild code PDF

```
не реагируем на собственный коммит бота
if: "!contains(github.event.head_commit.message, '[bot

steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
 run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommen
 # На Ubuntu политика ImageMagick иногда запре
 # разрешаем то, что нужно пайплайну (text -> p
 for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Step 1 – код → txt
 run: npm run export:txt

 - name: Step 2 – txt → png
 env:
 EXPORT DENSITY: ${github.event.inputs.densi
```

```
- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.*
 if git diff --cached --quiet; then
 echo "Экспорт не изменился – коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full_code_all_pages.*"
 git push origin "HEAD:${GITHUB_REF_NAME}"
```